

Contents

1	De secuencias homólogas a hipótesis evolutivas	1
1.1	La inferencia comienza con una pregunta	1
1.2	Leer árboles sin añadir historia que no contienen	3
1.2.1	Partes de un árbol	3
1.2.2	Ancastro común, clado y grupos hermanos	3
1.2.3	Rotar no es cambiar la historia	4
1.2.4	La raíz aporta polaridad	5
1.2.5	Politomías y ausencia de resolución	5
1.2.6	Newick: una topología escrita	6
1.3	Antes del alineamiento: ¿qué relación comparamos?	6
1.3.1	Similitud y homología no son sinónimos	6
1.3.2	Ortólogos y parálogos: la pregunta decide qué copia sirve	7
1.3.3	Árbol génico y árbol de especies	7
1.4	Homología posicional: el alineamiento es una hipótesis	7
1.4.1	Por qué dos alineamientos pueden ser plausibles	8
1.4.2	Alineamiento progresivo y árbol guía	8
1.4.3	¿Recortar siempre mejora?	9
1.4.4	Hueco no significa automáticamente un quinto nucleótido	10
1.5	De diferencias observadas a distancias de modelo	10
1.5.1	Distancia p: empezar por lo observable	10
1.5.2	Cambios múltiples y saturación	11
1.5.3	JC69: la corrección más simétrica	11
1.5.4	K2P: distinguir dos clases de cambio	12
1.5.5	De pares de secuencias a una matriz	13
1.6	UPGMA: una traza transparente y un supuesto fuerte	14
1.6.1	La regla de actualización	14
1.6.2	Ejemplo completo con cuatro taxones	14
1.6.3	Qué supone UPGMA	15
1.7	Unión de vecinos: distancias sin reloj estricto	16
1.8	De una matriz resumida a los caracteres	16
1.8.1	Cuántos árboles pueden proponerse	17
1.8.2	Parsimonia: contar cambios mínimos	18
1.8.3	Máxima verosimilitud: probabilidad de los datos bajo una hipótesis	19
1.8.4	Inferencia bayesiana: incorporar una distribución previa	19
1.8.5	Comparar familias sin convertirlas en una clasificación de calidad	20
1.9	Modelo, selección y herramienta	20
1.9.1	Un modelo es una aproximación, no un nombre de menú	20
1.9.2	Una tubería reproducible, no una receta ciega	21
1.10	Apoyo no significa verdad	22
1.10.1	Qué remuestrea el bootstrap	22

1.10.2	Apoyo local y estabilidad global	24
1.11	Sensibilidad: perturbar una decisión con intención	24
1.12	Errores y límites que un árbol bonito puede ocultar	25
1.12.1	Atracción de ramas largas	25
1.12.2	Poca señal y conflicto no son lo mismo	25
1.12.3	Composición, tasas y dependencia entre sitios	26
1.12.4	Recombinación y mosaicos de historia	26
1.12.5	Etiquetas y procedencia	27
1.13	Python como cuaderno verificable	27
1.13.1	Bibliotecas: conveniencia sin ocultar el objeto	28
1.14	Un caso integrado: de la pregunta a una afirmación limitada	28
1.15	Síntesis: qué puede defenderse y qué queda abierto	29
Anexo práctico · De secuencias a una hipótesis filogenética		31

Chapter 1

De secuencias homólogas a hipótesis evolutivas

1.1 La inferencia comienza con una pregunta

Imagina que recibes cuatro secuencias de una proteína presente en cuatro organismos. Dos de ellas se parecen mucho, una se parece algo menos y la cuarta es claramente diferente. Es tentador ordenar las secuencias de la más parecida a la menos parecida y llamar «filogenia» a ese orden. Sin embargo, una historia evolutiva no es una clasificación por parecido. Para defenderla necesitamos responder, al menos, a seis preguntas:

Esencial

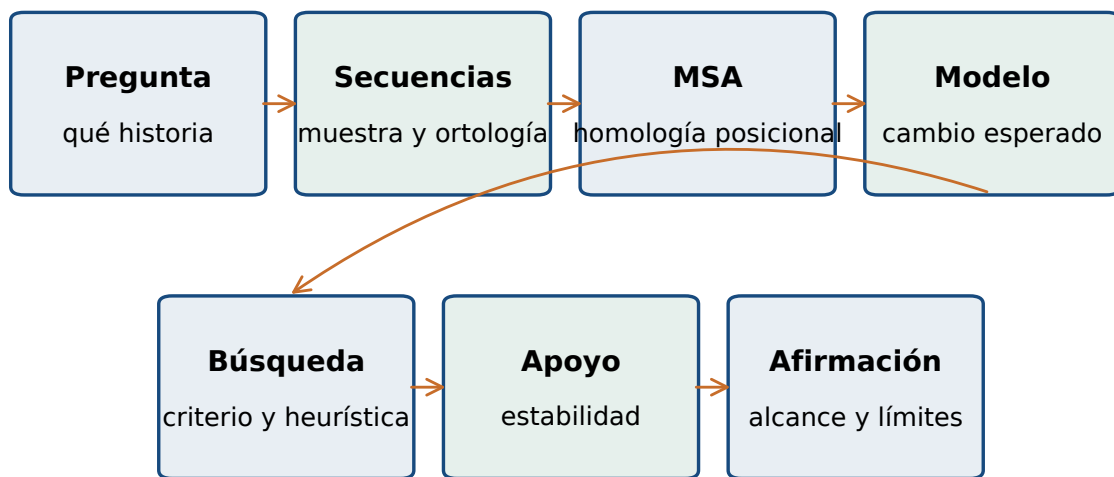
1. ¿estamos comparando copias que comparten origen evolutivo y cuya historia responde a nuestra pregunta?;
2. ¿qué posiciones de unas secuencias corresponden a qué posiciones de las otras?;
3. ¿cómo convertimos las diferencias observadas en una descripción del cambio que pudo ocurrir?;
4. ¿qué criterio empleamos para escoger entre historias alternativas?;
5. ¿cuánto depende el resultado del muestreo y de nuestras decisiones?;
6. ¿qué afirmación biológica permite realmente el análisis?

La respuesta no es una orden aislada de software. Es una cadena argumental. Las secuencias son datos observados; el alineamiento propone qué caracteres comparar; el modelo describe cómo pueden cambiar; el algoritmo explora árboles; el criterio puntúa las hipótesis; y el análisis de apoyo o sensibilidad examina su estabilidad. Un fallo temprano se propaga. Un conjunto con parálogos mal etiquetados no se vuelve adecuado porque el programa sea sofisticado, y un alineamiento ambiguo no se transforma en certeza al calcular mil réplicas.

Una conclusión filogenética es condicional a una cadena de decisiones que comienza antes del cálculo del árbol.

Tres rutas de lectura

La **ruta esencial** comprende lectura de árboles, homología, alineamiento, distancias, UPGMA, comparación de familias de métodos, bootstrap y límites. La **ruta de estudio** añade las trazas numéricas y los programas Python. La **ruta de referencia** profundiza en espacio de árboles, máxima verosimilitud, Bayes y discordancia entre árboles génicos y de especies. Las tres rutas comparten una tesis: un árbol es una hipótesis, no una fotografía del pasado.



Una decisión aguas arriba puede alterar las inferencias posteriores

Figure 1.1: Cadena de inferencia que separa pregunta, datos, alineamiento, modelo, búsqueda, apoyo y afirmación. Figura original generada por `verification/make_figures.py`.

Descripción alternativa. Cadena de siete etapas, desde la pregunta hasta una afirmación limitada. Las flechas indican dependencia aguas abajo: selección de secuencias, MSA, modelo, búsqueda y apoyo condicionan la conclusión.

La cifra contractual de 150–180 minutos para la ruta esencial queda superada por un presupuesto analítico reproducible de **188–264 minutos**. El artefacto cuenta 6.258 palabras en tramos esenciales, cuatro cálculos manuales y cinco listados; aplica 110–140 palabras por minuto, 12–18 minutos por cálculo, 10–15 por listado y 45–60 de consolidación. Son supuestos conservadores de planificación, no una medición de comprensión estudiantil. Un ensayo temporizado validará después el modelo; mientras tanto, profesorado y alumnado pueden ver y modificar cada componente en `verification/workload_budget.py`.

Con los supuestos declarados, el presupuesto analítico de la ruta esencial es 188–264 minutos para 6.258 palabras, cuatro cálculos manuales, cinco listados y consolidación.

1.2 Leer árboles sin añadir historia que no contienen

Esencial

1.2.1 Partes de un árbol

Un árbol es una estructura formada por *nodos* y *ramas*. En un árbol de secuencias, las hojas o nodos terminales representan las secuencias observadas. Los nodos internos representan divergencias hipotéticas, no secuencias que hayamos observado directamente. Dos nodos conectados por una rama mantienen una relación estructural; el significado cuantitativo de la longitud de esa rama depende de cómo se haya construido y dibujado el árbol.

En un árbol filogenético de secuencias, las hojas representan las secuencias observadas incluidas en el análisis.

Los nodos internos representan divergencias hipotéticas y no secuencias observadas directamente.

La topología de un árbol describe el patrón de conexiones entre nodos, independientemente del orden visual de las hojas y, en un cladograma, de la longitud con que se dibujen las ramas.

Conviene distinguir tres representaciones:

Cladograma. Solo la topología transmite información. Alargar una rama para que quepa una etiqueta no añade evolución.

Filograma. Las longitudes de rama representan una cantidad estimada de cambio, por ejemplo sustituciones por sitio.

Cronograma. Las longitudes se calibran en unidades de tiempo. Para construirlo hace falta información y modelización adicionales.

Representación	Qué codifica una rama	Qué no debe inferirse sin datos extra
Cladograma	relación de descendencia	cantidad de cambio o tiempo
Filograma	cambio estimado bajo un modelo	tiempo absoluto
Cronograma	tiempo estimado y calibrado	certeza histórica de la topología

Un cladograma codifica topología; un filograma añade cambio estimado en las longitudes y un cronograma añade tiempo calibrado. La longitud gráfica solo se interpreta después de identificar representación y escala.

1.2.2 Ancestro común, clado y grupos hermanos

En un árbol enraizado, el *ancestro común más reciente* de dos hojas es el primer nodo que encontramos al retroceder desde ambas hacia la raíz. Un *clado* contiene un ancestro y todos

sus descendientes representados. Dos linajes que parten del mismo nodo son grupos hermanos. Estas definiciones evitan una lectura basada en la distancia horizontal del papel.

El ancestro común más reciente de dos hojas de un árbol enraizado es el primer nodo compartido al recorrer ambas hacia la raíz.

En un árbol enraizado, un clado contiene un ancestro y todos sus descendientes representados.

Dos linajes que parten del mismo nodo son grupos hermanos.

Supongamos que la topología es $((A,B),(C,D))$; . El ancestro común más reciente de A y B es el nodo que une únicamente A y B. El ancestro común más reciente de A y C es la raíz, que también tiene como descendientes a B y D. Por tanto, A y B forman un clado; A y C no forman por sí solos un clado. Que A aparezca en la primera línea y C en la tercera no interviene en el razonamiento.

1.2.3 Rotar no es cambiar la historia

Una rama interna puede imaginarse como el eje de un móvil colgante. Podemos intercambiar el orden de sus descendientes sin cortar ninguna conexión. El dibujo cambia; los conjuntos de descendientes no.

Tres dibujos, los mismos clados: $\{A,B\}$ y $\{C,D\}$

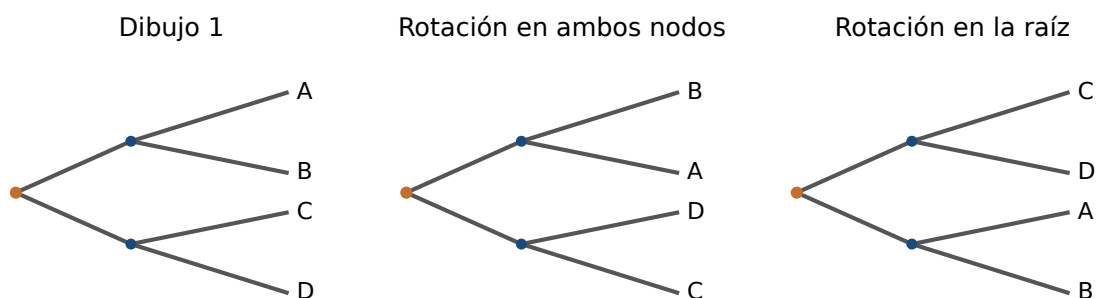


Figure 1.2: Tres representaciones de la misma topología enraizada. En todos los paneles permanecen los clados $\{A,B\}$ y $\{C,D\}$.

Descripción alternativa. Tres rotaciones de una misma topología. En las tres, A y B comparten un nodo y C y D comparten otro; solo cambia el orden gráfico de los descendientes.

Rotar las ramas hijas alrededor de un nodo interno no cambia los clados representados por un árbol enraizado.

Este principio corrige el llamado razonamiento en escalera: leer de izquierda a derecha o de arriba abajo como si las especies actuales fueran etapas de una progresión. En un árbol de especies actuales, ninguna hoja es antepasada de otra solo por ocupar una posición «inferior» o «temprana» en el dibujo [22, 21]. Si peces y mamíferos son hojas, ambos han evolucionado desde su ancestro común durante el tiempo transcurrido; el pez actual no es un borrador del mamífero actual.

La posición gráfica de una hoja no la convierte en antepasado de otra hoja: leer especies actuales como peldaños de progreso es una interpretación errónea del patrón de ramificación.

1.2.4 La raíz aporta polaridad

Un árbol no enraizado expresa separaciones o *splits*. Por ejemplo, $AB|CD$ indica que una rama separa A y B de C y D. No dice por sí sola dónde comenzó la historia. Colocar la raíz en diferentes ramas puede cambiar qué grupos interpretamos como hermanos y en qué orden inferimos las divergencias.

Un árbol no enraizado expresa una bipartición o split, por ejemplo $AB|CD$, sin indicar dónde comenzó la historia; enraizarlo en ramas distintas puede cambiar qué grupos se interpretan como hermanos.

La raíz añade polaridad; no aparece por mover el dibujo

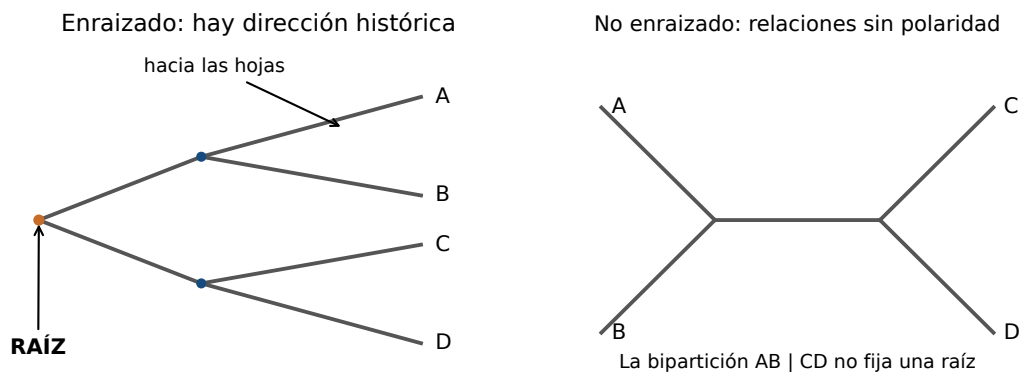


Figure 1.3: Un árbol enraizado contiene polaridad histórica; el árbol no enraizado solo fija la bipartición mostrada.

Descripción alternativa. El panel enraizado contiene un origen explícito y polaridad; el no enraizado conserva el split AB partido CD pero no indica de qué rama procede la historia.

La raíz polariza un árbol: permite interpretar dirección ancestro–descendiente, pero no se deduce de la orientación de la página.

Una estrategia habitual es incluir un *grupo externo*: una secuencia que queda fuera del grupo cuya historia queremos estudiar y cuya posición externa se justifica con conocimiento independiente. No basta con escoger la secuencia más diferente. Si es demasiado distante, el alineamiento puede ser incierto, la saturación mayor y el enraizamiento vulnerable a artefactos. El enraizamiento por punto medio coloca la raíz a mitad del camino más largo entre dos puntas; resulta útil como recurso exploratorio, pero depende implícitamente de que las distancias desde la raíz sean suficientemente comparables.

El enraizamiento con grupo externo requiere justificar con evidencia independiente que ese grupo queda fuera del grupo focal.

Elegir como grupo externo solo la secuencia más diferente puede agravar incertidumbre de alineamiento y saturación.

El enraizamiento por punto medio es exploratorio y supone que las distancias desde la raíz son suficientemente comparables; no aporta por sí solo evidencia externa de polaridad.

1.2.5 Politomías y ausencia de resolución

Un nodo con más de dos descendientes es una politomía. Puede expresar una divergencia realmente cercana en el tiempo o, con más frecuencia en un árbol estimado, que los datos o el análisis no resuelven el orden de separación. Convertir automáticamente toda politomía en una

serie de bifurcaciones produce precisión gráfica, no información. Una buena lectura distingue «el análisis apoya una divergencia simultánea» de «el análisis no permite decidir entre varias bifurcaciones».

Una politomía en un árbol estimado puede representar una divergencia cercana o falta de resolución; convertirla sin evidencia en bifurcaciones añade precisión gráfica, no información.

1.2.6 Newick: una topología escrita

El formato Newick escribe paréntesis anidados. Por ejemplo:

```
((A:0.1,B:0.1):0.2,
 (C:0.15,D:0.15):0.15);
```

La cadena contiene dos parejas y longitudes después de los dos puntos. La coma separa hijos del mismo nodo; los paréntesis delimitan un grupo; el punto y coma cierra el árbol.

En Newick, los paréntesis delimitan descendientes, la coma separa hijos, los dos puntos preceden longitudes y el punto y coma cierra el árbol; el orden textual de hermanos puede variar sin cambiar la topología.

Python nos permite representar clados sin instalar ninguna biblioteca. Una *lista* conserva elementos en orden y se escribe entre corchetes; un *conjunto* conserva pertenencia sin imponer orden y se escribe entre llaves. Para comprobar la rotación usamos conjuntos porque nos importa quién pertenece al clado, no dónde se imprime. El bloque es completo y solo usa la biblioteca estándar:

```
clados_dibujo_1 = {frozenset({"A", "B"}),
                    frozenset({"C", "D"})}
clados_dibujo_2 = {frozenset({"B", "A"}),
                    frozenset({"D", "C"})}

assert clados_dibujo_1 == clados_dibujo_2
```

`frozenset` crea un conjunto que ya no se modifica y que, por ello, puede incluirse dentro de otro conjunto. `assert` exige que una condición sea verdadera; si no lo es, Python detiene el programa. No «prueba» por sí solo una teoría biológica, pero sí evita que una rotación accidental se interprete como una topología distinta.

1.3 Antes del alineamiento: ¿qué relación comparamos?

1.3.1 Similitud y homología no son sinónimos

La *similitud* es una observación o una medida: dos secuencias comparten cierta proporción de símbolos, una puntuación o un patrón. La *homología* es una relación histórica: dos caracteres son homólogos si derivan de un carácter ancestral común. Por ello, la similitud aporta evidencia para proponer homología, pero no la demuestra de forma aislada. También puede existir homología con similitud baja después de mucha divergencia.

La homología es una relación de origen común, mientras que la similitud es una propiedad observable o cuantificable que puede aportar evidencia, pero no equivale lógicamente a homología.

La expresión «80 % de homología» confunde ambos niveles. Puede decirse «80 % de identidad en este alineamiento» o «se propone que son homólogos», pero no que la relación histórica sea verdadera en un porcentaje.

La *homoplasia* es una semejanza que no se explica por herencia directa del mismo estado desde el ancestro relevante. Puede surgir por convergencia, por evolución paralela o por una reversión. En secuencias de cuatro símbolos, los mismos estados pueden aparecer varias

veces por azar o por restricciones funcionales; por eso las similitudes individuales no se leen automáticamente como sinapomorfías.

La homoplasia es una semejanza no explicada por la herencia directa del mismo estado ancestral en el nodo relevante y puede surgir por convergencia, evolución paralela o reversión.

En secuencias de cuatro símbolos, un mismo estado puede repetirse por azar o por restricciones funcionales, por lo que una similitud individual no se lee automáticamente como sinapomorfía sin evaluar el contexto filogenético.

1.3.2 Ortólogos y parálogos: la pregunta decide qué copia sirve

Dos genes homólogos separados por un evento de especiación son *ortólogos*; dos genes homólogos separados por una duplicación son *parálogos*. Son relaciones entre pares de copias, no propiedades permanentes de un nombre de gen. Tras una duplicación ancestral, una copia de la especie A puede ser ortóloga de la copia correspondiente en B y paróloga de otra copia dentro de A.

Ortología y paralogía distinguen homologías según el evento que separa las copias: especiación o duplicación, respectivamente [8].

Considera una duplicación que produjo globina alfa y globina beta antes de que se separaran humano y ratón. La alfa humana y la alfa de ratón son ortólogas respecto a esa especiación. La alfa humana y la beta humana son parálogas respecto a la duplicación. Si mezclamos alfa de unas especies, beta de otras y mioglobina de una tercera sin declarar la pregunta, el árbol resultante puede ser una historia de una familia génica, pero no un árbol directo de especies.

1.3.3 Árbol génico y árbol de especies

Un árbol génico representa la genealogía de copias de un locus. Un árbol de especies representa relaciones entre linajes de organismos bajo una definición y un modelo determinados. No son objetos intercambiables. Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real. Además, un árbol génico estimado puede diferir por error de muestreo, alineamiento, modelo o búsqueda [18, 17].

Referencia

Duplicación y pérdida, transferencia horizontal, hibridación, introgresión y clasificación incompleta de linajes pueden producir discordancia real entre el árbol génico y el árbol de especies.

Un árbol génico estimado puede diferir del árbol génico verdadero por error de muestreo, alineamiento, modelo o búsqueda, con independencia de que exista discordancia biológica real.

Un árbol de un solo gen no se identifica automáticamente con el árbol de especies, porque procesos biológicos y errores de inferencia pueden separar ambas historias.

Esta distinción cambia la selección de datos. Para estudiar una familia multigénica, la duplicación es parte del fenómeno y debemos conservar copias parálogas bien anotadas. Para aproximar relaciones entre especies con un locus, intentamos comparar ortólogos y dejamos claro que el resultado sigue siendo un árbol génico con posible discordancia.

Para estudiar una familia multigénica se conservan copias parálogas bien anotadas porque la duplicación es parte del fenómeno; para aproximar relaciones entre especies con un locus se prefiere comparar ortólogos, sabiendo que el resultado sigue siendo un árbol génico con posible discordancia.

1.4 Homología posicional: el alineamiento es una hipótesis

Una inferencia basada en caracteres compara columnas. Antes de contar cambios necesitamos decidir qué nucleótidos o aminoácidos ocupan posiciones comparables. Un alineamiento

Esencial

La historia de un gen no se identifica automáticamente con la historia de las especies

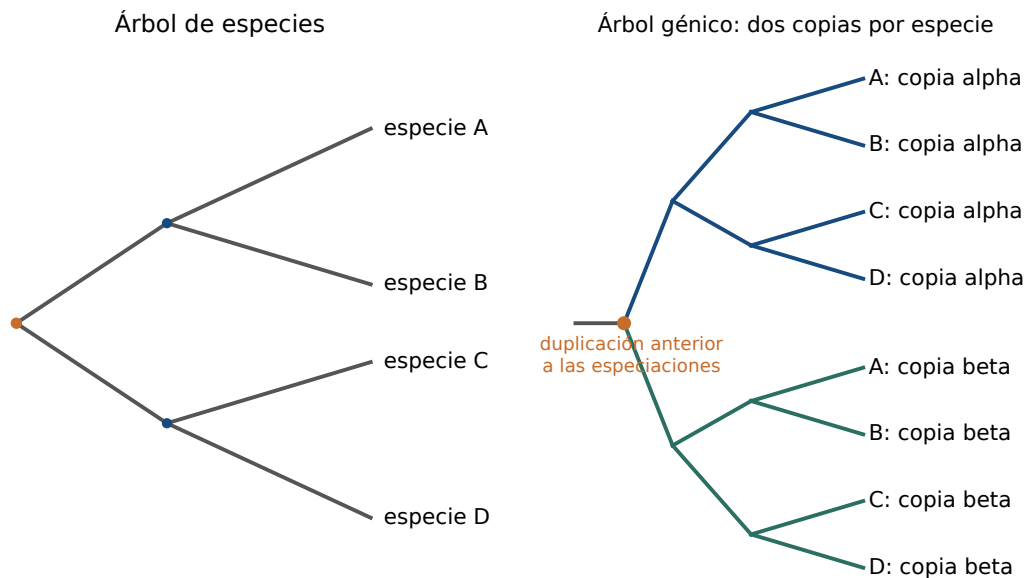


Figure 1.4: Contraste elemental entre un árbol de especies y un árbol génico afectado por una duplicación anterior a la especiación.

Descripción alternativa. El mismo universo A–D aparece en ambos paneles. La duplicación ancestral separa dos subárboles, alfa y beta, y cada uno conserva copias de A, B, C y D.

múltiple de secuencias, o MSA, inserta huecos y organiza los símbolos en columnas para proponer correspondencias.

Un alineamiento formula una hipótesis explícita de homología posicional; no observa directamente la historia de cada posición.

1.4.1 Por qué dos alineamientos pueden ser plausibles

Si una secuencia presenta una inserción y otra una delección, la posición exacta del evento puede ser ambigua, especialmente en regiones repetitivas. Dos alineamientos pueden obtener puntuaciones parecidas pero colocar el hueco en columnas distintas. Entonces cambian los patrones que verá el método filogenético.

No existe una regla universal que resuelva toda ambigüedad. La información estructural, la traducción de regiones codificantes, la conservación de motivos, la elección de algoritmo y los parámetros de hueco pueden ayudar. Para regiones codificantes suele ser útil alinear proteínas y proyectar la correspondencia a codones, porque insertar un solo nucleótido dentro de un codón puede romper el marco de lectura.

En regiones codificantes, alinear proteínas y proyectar la correspondencia a codones puede conservar el marco de lectura mejor que tratar cada nucleótido sin contexto, pero sigue siendo una hipótesis de alineamiento.

1.4.2 Alineamiento progresivo y árbol guía

Muchos alineadores construyen primero similitudes por pares, obtienen un árbol guía y alinean progresivamente secuencias o perfiles. El árbol guía organiza la operación; no debe confundirse con el árbol filogenético final. Las decisiones tempranas pueden condicionar las posteriores, y

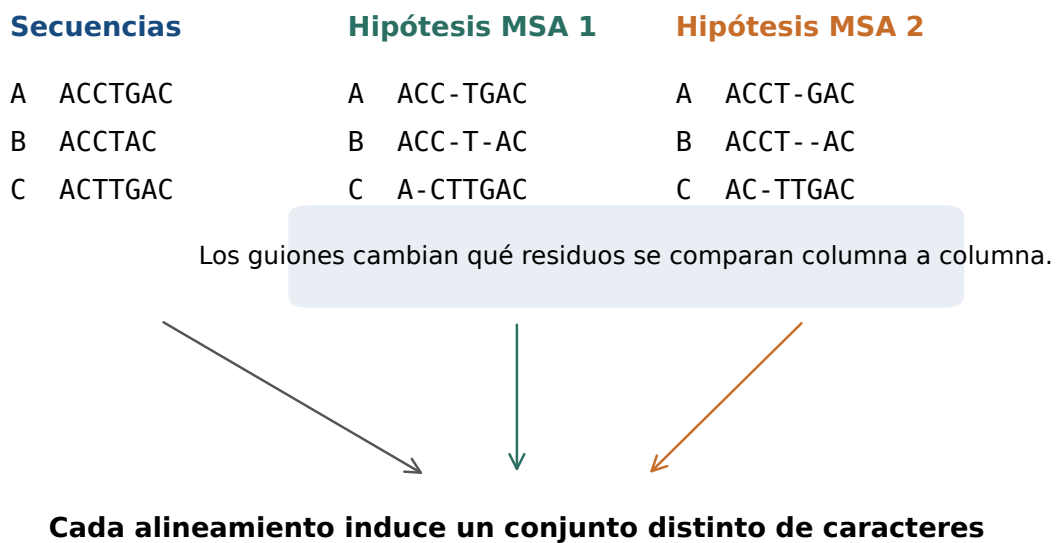


Figure 1.5: Dos hipótesis de homología posicional para secuencias cortas. Los símbolos son originales y solo ilustran la dependencia del análisis.

Descripción alternativa. Las dos matrices tienen ocho columnas en todas sus filas. En ambas, eliminar guiones recupera A igual a ACCTGAC, B igual a ACCTAC y C igual a ACTTGAC. Los guiones ocupan columnas distintas y por eso cambian los caracteres comparados.

las estrategias iterativas tratan de refinar el resultado. MAFFT ofrece distintas estrategias con compromisos de escala, velocidad y precisión [15]; la documentación oficial debe consultarse para la versión realmente usada.

En un esquema de alineamiento progresivo, el árbol guía fija el orden de unión de secuencias o perfiles.

En un esquema progresivo, el resultado de una unión temprana se convierte en entrada de las uniones posteriores.

El árbol guía de un alineamiento progresivo no es el árbol filogenético final.

Una salida reproducible no registra solo «se usó MAFFT». Debe conservar versión, orden exacta, parámetros, secuencias de entrada y huellas criptográficas. Cambiar de una estrategia automática a otra más intensiva es una perturbación analítica, no una mejora garantizada.

Una salida de alineamiento reproducible conserva versión del programa, orden exacto de las secuencias de entrada, parámetros y huellas criptográficas; cambiar de una estrategia automática a otra más intensiva es una perturbación analítica y no una mejora garantizada.

1.4.3 ¿Recortar siempre mejora?

Las regiones con muchos huecos o correspondencia dudosa pueden introducir ruido. Sin embargo, eliminarlas también puede borrar señal. Estudios comparativos han encontrado situaciones en las que el filtrado automático empeora árboles de un solo gen [26]. Por tanto, «alineamiento recortado» no significa «alineamiento verdadero».

El recorte de un MSA intercambia posible reducción de ruido por pérdida de caracteres; su efecto debe evaluarse y no asumirse siempre beneficioso.

Una comprobación docente útil conserva dos análisis: sin recorte y con una regla declarada. Primero se cuentan longitud, huecos y sitios variables; después se comparan topología y apoyo. Si desaparece un clado, la conclusión correcta no es escoger silenciosamente la versión preferida,

sino registrar que la inferencia es sensible al tratamiento del alineamiento.

1.4.4 Hueco no significa automáticamente un quinto nucleótido

Un guion en un MSA representa una ausencia introducida para proponer correspondencia. Puede relacionarse con una inserción, una delección, una región no secuenciada o una decisión del algoritmo. Diferentes programas pueden tratarlo como dato ausente, como estado adicional o mediante modelos de inserción y delección. Antes de calcular distancias hay que declarar si las columnas con huecos se excluyen para cada par, para todas las secuencias o de otra manera. Dos matrices calculadas con reglas distintas no son directamente comparables.

Tratar huecos mediante exclusión por pareja, exclusión completa, estado adicional o modelos de indel cambia qué columnas contribuyen; la política afecta al resultado calculado.

Dos matrices de distancia calculadas con políticas de huecos distintas no son directamente comparables como si resumieran los mismos caracteres.

Un hueco de un MSA no es automáticamente un quinto nucleótido: puede representar ausencia de dato, una hipótesis de inserción o delección u otro tratamiento que debe declararse.

1.5 De diferencias observadas a distancias de modelo

1.5.1 Distancia p: empezar por lo observable

Si dos secuencias alineadas no vacías tienen $L > 0$ columnas comparables y difieren en m de ellas, la distancia observada es

$$p = \frac{m}{L}, \quad 0 \leq m \leq L. \quad (1.1)$$

La distancia p es la proporción m/L de columnas discordantes entre las L columnas comparables de dos secuencias alineadas no vacías.

Es una proporción, no un porcentaje de homología ni una estimación completa de todos los cambios históricos.

En dos secuencias alineadas de 20 nucleótidos con dos diferencias válidas, la distancia observada es $p = 2/20 = 0.10$.

Tomemos este programa completo. Las comillas convierten cada secuencia en una cadena de Python antes de usarla:

```
S1 = "ACGTACGTACGTACGTACGT"
S2 = "ACGTACGGACGTACGGACGT"

def p_distance(seq1, seq2):
    if len(seq1) != len(seq2):
        raise ValueError("las secuencias deben estar alineadas")
    if len(seq1) == 0:
        raise ValueError("el alineamiento no puede estar vacio")

    differences = 0
    for base1, base2 in zip(seq1, seq2):
        if base1 != base2:
            differences = differences + 1
    return differences / len(seq1)

assert abs(p_distance(S1, S2) - 0.10) < 1e-12
```

La implementación Python de distancia p cuenta discordancias, divide por la longitud y rechaza secuencias desalineadas o vacías.

Difieren en las posiciones 8 y 16 si numeramos desde 1. En ambos casos T cambia a G, una transversión. Hay cero transiciones y dos transversiones: $P = 0/20 = 0$, $Q = 2/20 = 0,10$ y $p = P + Q = 0,10$.

Una *función* agrupa una operación y recibe sus entradas entre paréntesis. **zip** empareja el primer símbolo de una secuencia con el primero de la otra, y así sucesivamente. El bucle **for** visita cada par.

len devuelve la longitud. **!=** significa «distinto de». **raise ValueError** detiene el cálculo con un mensaje si la condición de entrada no se cumple. La última línea tolera un error de redondeo menor que 10^{-12} , porque muchos decimales se representan de forma aproximada en el ordenador.

1.5.2 Cambios múltiples y saturación

La distancia p solo observa el estado inicial y final de cada comparación. Si en una rama ocurrió $A \rightarrow G \rightarrow A$, las secuencias finales coinciden y el conteo observado es cero, aunque hubo dos sustituciones. Con mayor divergencia crece la posibilidad de cambios múltiples, reversión o convergencia. La distancia observada tiende entonces a subestimar el número de sustituciones.

La saturación surge cuando sustituciones múltiples ocultan cambios anteriores, de modo que la proporción observada de diferencias deja de crecer en correspondencia directa con el cambio acumulado.

Un modelo de sustitución no recupera el pasado exacto. Describe probabilísticamente el proceso y permite corregir la expectativa bajo supuestos explícitos. La corrección es útil precisamente porque revela sus límites: cuando los datos se acercan al equilibrio del modelo, la incertidumbre crece y pequeñas variaciones de p producen grandes cambios en la distancia.

1.5.3 JC69: la corrección más simétrica

JC69 supone frecuencias de equilibrio iguales para A, C, G y T y la misma tasa instantánea para cada cambio entre bases distintas [13]. No es «la evolución real», sino un modelo mínimo para comprender por qué una corrección es no lineal.

JC69 supone frecuencias de equilibrio iguales para los cuatro nucleótidos y la misma tasa instantánea para cada sustitución entre bases distintas.

$$d_{\text{JC69}} = -\frac{3}{4} \ln\left(1 - \frac{4p}{3}\right), \quad 0 \leq p < \frac{3}{4}. \quad (1.2)$$

La corrección JC69 tiene una distancia real finita solo para una proporción observada $p < 3/4$.

El dominio no es un detalle tipográfico. Para $p = 0,75$, el argumento del logaritmo es cero y la distancia diverge; para valores mayores es negativo y no hay distancia real bajo esta expresión. Informar un número finito en esa frontera sería un error. Además, estar justo por debajo no vuelve fiable la estimación: la curva ya es muy pronunciada.

Para $p = 0,10$, los pasos son:

$$1 - \frac{4(0,10)}{3} = 0,866666\dots, \quad \ln(0,866666\dots) \approx -0,143101,$$

$$d = -0,75(-0,143101) \approx 0,107326.$$

Para una proporción observada de diferencias $p = 0,1$, JC69 produce aproximadamente 0,107326 sustituciones por sitio.

La corrección es ligeramente mayor que 0,10 porque admite que algunos cambios no quedaron visibles. El siguiente programa reexplica cada construcción:

```

import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    inside = 1 - 4 * p / 3
    return -3 / 4 * math.log(inside)

distance = jc69(0.10)
print(f"{distance:.6f}") # imprime 0.107326
assert abs(distance - 0.107326) < 1e-6

```

`import math` hace disponible el módulo matemático de la biblioteca estándar. `math.log` calcula el logaritmo natural. La expresión `f"{distance:.6f}"` solicita seis decimales; no cambia el valor interno, solo su presentación.

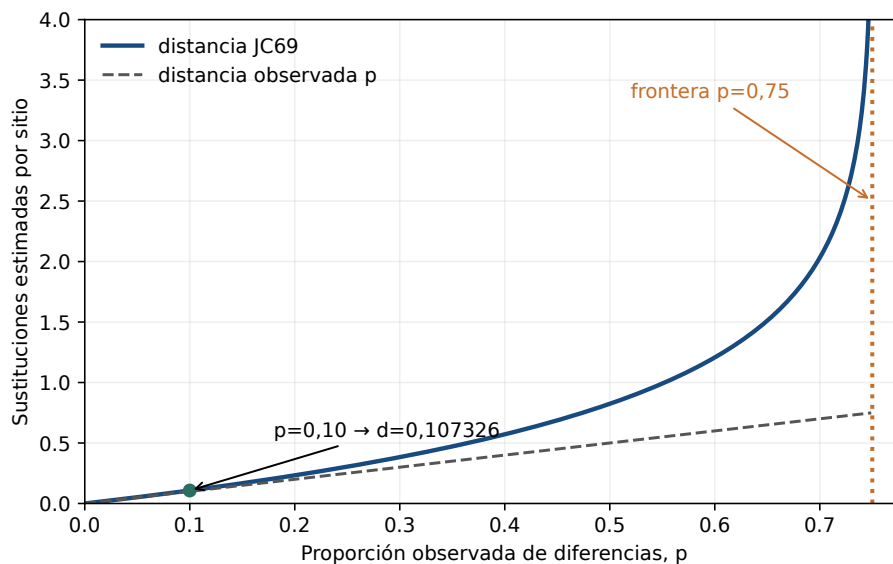


Figure 1.6: Distancia observada y corrección JC69. La divergencia cerca de $p = 0,75$ señala la pérdida de información bajo el modelo.

Descripción alternativa. La corrección JC69 coincide aproximadamente con p cerca de cero, luego crece más deprisa y diverge al aproximarse a la frontera p igual a 0,75.

1.5.4 K2P: distinguir dos clases de cambio

Las transiciones intercambian purinas entre sí ($A \leftrightarrow G$) o pirimidinas entre sí ($C \leftrightarrow T$). Las transversiones cambian entre purina y pirimidina. K2P permite que ambas clases tengan tasas distintas [16]. Sea P la proporción de transiciones y Q la de transversiones:

Entre nucleótidos distintos, A–G y C–T son transiciones; los otros ocho cambios dirigidos entre purina y pirimidina son transversiones.

$$d_{K2P} = -\frac{1}{2} \ln(1 - 2P - Q) - \frac{1}{4} \ln(1 - 2Q). \quad (1.3)$$

K2P tiene una distancia real finita cuando P y Q son proporciones válidas y se cumplen simultáneamente $1 - 2P - Q > 0$ y $1 - 2Q > 0$.

Las desigualdades son parte del dominio. En la frontera $1 - 2P - Q = 0$ o $1 - 2Q = 0$, un logaritmo recibe cero; más allá recibe un número negativo. En ambos casos no existe una distancia real finita. Además, $P \geq 0$, $Q \geq 0$ y $P + Q \leq 1$ porque son proporciones de sitios.

Para un segundo ejemplo, supongamos $P = 0,08$ y $Q = 0,02$. Entonces $1 - 2P - Q = 0,82$ y $1 - 2Q = 0,96$. Sustituimos:

$$d = -\frac{1}{2} \ln(0,82) - \frac{1}{4} \ln(0,96) \approx 0,099225 + 0,010205 = 0,109431.$$

Para $P = 0,08$ transiciones y $Q = 0,02$ transversiones, K2P produce aproximadamente 0,109431 sustituciones por sitio.

El cálculo no demuestra que K2P sea adecuado. Solo garantiza que hemos aplicado correctamente la fórmula. La adecuación exige preguntar si separar esas dos clases captura suficientemente el patrón, si la composición es comparable y si las tasas varían entre sitios o linajes. Modelos más flexibles incorporan frecuencias desiguales, tasas diferentes y heterogeneidad, pero cada parámetro adicional debe estimarse.

Aumentar la flexibilidad de un modelo permite representar más patrones mediante parámetros adicionales que deben estimarse, pero no demuestra por sí solo que el modelo sea adecuado para la pregunta.

1.5.5 De pares de secuencias a una matriz

Con n secuencias calculamos una distancia para cada par y obtenemos una matriz simétrica. La diagonal vale cero, porque cada secuencia se compara consigo misma. Los valores deben ser no negativos y las etiquetas de filas y columnas deben coincidir.

	A	B	C	D
A	0	17	21	31
B	17	0	30	34
C	21	30	0	28
D	31	34	28	0

La matriz reconstruida del ejemplo contiene cuatro taxones, es simétrica, tiene diagonal nula y no contiene distancias negativas.

En Python, un *diccionario* asocia una clave con un valor. Podemos usar un diccionario de diccionarios para acceder a `distance["A"]["B"]`. Las comillas indican una cadena de texto. Esta representación hace visibles las etiquetas y reduce el riesgo de confundir posiciones:

```
distance = {
    "A": {"A": 0, "B": 17, "C": 21, "D": 31},
    "B": {"A": 17, "B": 0, "C": 30, "D": 34},
    "C": {"A": 21, "B": 30, "C": 0, "D": 28},
    "D": {"A": 31, "B": 34, "C": 28, "D": 0},
}

taxa = sorted(distance)
for a in taxa:
    assert distance[a][a] == 0
    for b in taxa:
        assert distance[a][b] == distance[b][a]
        assert distance[a][b] >= 0
```

`sorted` crea una lista ordenada de claves. Los dos bucles recorren todas las parejas, también la diagonal. Una matriz que declara siete taxones en la cabecera y contiene ocho filas fallaría antes de producir un árbol; ese es el tipo de control que debe preceder al análisis.

1.6 UPGMA: una traza transparente y un supuesto fuerte

UPGMA es un algoritmo aglomerativo. Empieza con cada taxón como clúster individual, une la pareja más cercana, actualiza las distancias y repite hasta formar un solo árbol. Su valor didáctico es extraordinario porque cada decisión puede comprobarse. Su limitación también es clara: interpreta las distancias bajo un reloj suficientemente uniforme y produce un árbol ultramétrico.

UPGMA comienza con clústeres individuales, une en cada paso una pareja de distancia mínima, actualiza distancias ponderando tamaños y repite hasta obtener un árbol enraizado ultramétrico.

1.6.1 La regla de actualización

Si se unen los clústeres X e Y para formar $U = X \cup Y$, la distancia desde U hasta otro clúster Z es la media de todas las parejas de hojas. Por ello debe ponderarse por el número de elementos:

$$d(U, Z) = \frac{|X| d(X, Z) + |Y| d(Y, Z)}{|X| + |Y|}. \quad (1.4)$$

La actualización de UPGMA pondera cada distancia de clúster por su número de hojas; una media no ponderada solo coincide cuando los clústeres tienen igual tamaño.

1.6.2 Ejemplo completo con cuatro taxones

Partimos de la matriz anterior. La menor distancia es $d(A, B) = 17$, así que unimos A y B. El nodo queda a altura $17/2 = 8,5$. Como ambos clústeres tenían una hoja:

$$d(AB, C) = \frac{21 + 30}{2} = 25,5, \quad d(AB, D) = \frac{31 + 34}{2} = 32,5.$$

La primera reducción del ejemplo UPGMA produce $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y altura $h(AB) = 8,5$.

La matriz reducida contiene $d(AB, C) = 25,5$, $d(AB, D) = 32,5$ y $d(C, D) = 28$. La menor distancia es 25,5; unimos el clúster AB con C. El nuevo nodo queda a altura $25,5/2 = 12,75$. La rama desde el nodo AB mide $12,75 - 8,5 = 4,25$; la rama hasta C mide 12,75.

Ahora aparece el paso que una media ingenua resuelve mal. El clúster AB tiene dos hojas y C tiene una. La distancia a D es:

$$d(ABC, D) = \frac{2 d(AB, D) + 1 d(C, D)}{2 + 1} = \frac{2(32,5) + 28}{3} = 31.$$

Al combinar un clúster de tamaño dos situado a distancia 32,5 y un singleton a distancia 28, la actualización ponderada de UPGMA es 31.

La raíz queda a altura $31/2 = 15,5$. La rama desde ABC hasta la raíz mide $15,5 - 12,75 = 2,75$, y la rama desde D mide 15,5. El árbol Newick, incluyendo longitudes, es:

```
((A:8.5,B:8.5):4.25,C:12.75):2.75,D:15.5);
```

La cadena Newick del ejemplo codifica las fusiones A, B , después $(AB), C$, y longitudes que dejan las cuatro puntas a distancia 15,5 de la raíz.

El mismo cálculo en Python mantiene el tamaño del clúster de forma explícita:

```
size_ab = 2
size_c = 1
distance_ab_d = 32.5
distance_c_d = 28.0
```

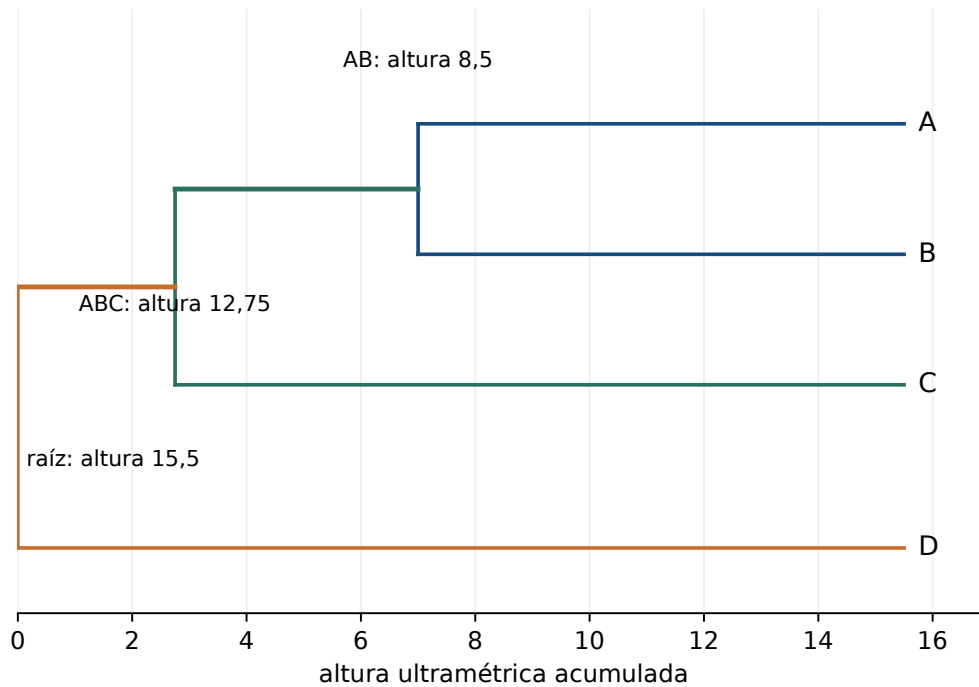



Figure 1.7: Dendrograma UPGMA reconstruido a partir de la matriz de cuatro taxones. Todas las puntas quedan a altura 15,5.

Descripción alternativa. La raíz se sitúa a la izquierda; A, B, C y D aparecen en los cuatro extremos derechos, todos a 15,5. A y B se fusionan a altura 8,5; AB con C a 12,75; y ABC con D a 15,5.

```
distance_abc_d = (
    size_ab * distance_ab_d + size_c * distance_c_d
) / (size_ab + size_c)

assert abs(distance_abc_d - 31.0) < 1e-12
```

Los paréntesis agrupan una expresión que ocupa varias líneas. Los nombres descriptivos evitan el error de confundir distancia con altura. El programa no debe esconder el factor 2: es la memoria de que AB representa dos hojas.

1.6.3 Qué supone UPGMA

UPGMA produce un árbol ultramétrico y requiere que las distancias sean compatibles con tasas suficientemente constantes desde la raíz; si los linajes evolucionan a velocidades distintas puede agrupar por tasa en lugar de por historia.

Ser ultramétrico significa que todas las hojas quedan a la misma distancia de la raíz. El *árbol producido* por el ejemplo fuerza que cada camino sume 15,5, pero la matriz de entrada no es ultramétrica. En una matriz ultramétrica, las dos distancias mayores de cada terna deben coincidir. Aquí las cuatro ternas violan esa propiedad; por ejemplo, para A, B y C aparecen 17, 21 y 30.

El ajuste UPGMA implica distancias cophenéticas 17 para AB; 25,5 para AC y BC; y 31 para AD, BD y CD. Al restar distancia de entrada, los residuos son:

Par	AB	AC	BC	AD	BD	CD
cophenética – entrada	0	+4,5	-4,5	0	-3	+3

La matriz del ejemplo viola la ultrametricidad en sus cuatro ternas y el árbol UPGMA

presenta residuos cophenéticos 0, +4,5, -4,5, 0, -3, +3 para AB, AC, BC, AD, BD y CD.

La salida ultramétrica no demuestra, por tanto, que la entrada sostuviera un reloj. Un reloj estricto presupone que las secuencias se muestrean al mismo tiempo y el cambio esperado es proporcional al tiempo compartido. Si un linaje acumula sustituciones mucho más rápido, la distancia ya no funciona como reloj común y la jerarquía de UPGMA puede ser incorrecta.

UPGMA no es «malo» por tener un supuesto fuerte. Es apropiado cuando el supuesto se sostiene suficientemente para la pregunta, y muy útil para aprender a separar datos, regla y resultado. El error consiste en usarlo sin reconocer el supuesto o en generalizar su árbol a cualquier matriz.

UPGMA es apropiado cuando su supuesto de reloj se sostiene suficientemente para la pregunta planteada; el error no es el supuesto en sí, sino usarlo sin reconocerlo o generalizar su árbol a cualquier matriz.

1.7 Unión de vecinos: distancias sin reloj estricto

Neighbor joining, o unión de vecinos (NJ), también parte de una matriz de distancias, pero no escoge simplemente la menor entrada. Busca una pareja cuya unión reduzca la longitud total estimada del árbol, corrigiendo la decisión por la separación de cada taxón respecto al conjunto [24]. Comienza con una estrella no resuelta, une vecinos, calcula longitudes de rama y reduce la matriz hasta obtener un árbol no enraizado.

Neighbor joining construye un árbol a partir de distancias sin requerir un reloj molecular estricto; no debe describirse como una variante de UPGMA con reloj relajado.

La diferencia conceptual puede verse con una matriz en la que todos los taxones no están a igual distancia de una raíz posible. UPGMA obliga a que las puntas terminen a la misma altura. NJ puede asignar ramas terminales diferentes. Eso no convierte automáticamente a NJ en la mejor explicación: su resultado depende de que las distancias representen adecuadamente el proceso y resume cada par de secuencias en un número.

Que NJ asigne ramas terminales distintas de UPGMA no lo convierte automáticamente en la mejor explicación: su resultado sigue dependiendo de que las distancias representen adecuadamente el proceso evolutivo.

Para n taxones, NJ emplea la matriz auxiliar

$$Q(i, j) = (n - 2)d(i, j) - \sum_k d(i, k) - \sum_k d(j, k).$$

La pareja con menor Q , no necesariamente con menor d , se une en ese paso. NJ calcula $Q(i, j)$ mediante las sumas de fila y une en cada iteración una pareja que minimiza Q ; para la matriz del ejemplo, AB y CD empatan en el mínimo -116.

La ecuación compensa que un taxón muy alejado de todos los demás pueda tener una distancia par engañosamente pequeña o grande. En este capítulo no desarrollamos toda la traza de NJ: el ejemplo completamente trabajado es UPGMA. Lo importante es conservar tres diferencias:

1. ambos métodos usan distancias y pierden los patrones de cada columna;
2. UPGMA produce un árbol enraizado ultramétrico;
3. NJ produce inicialmente un árbol no enraizado y no exige reloj.

1.8 De una matriz resumida a los caracteres

Los métodos de distancias transforman un MSA de muchas columnas en una matriz de números por pares. Los métodos basados en caracteres conservan el patrón de cada columna. Esto

permite preguntar qué sitios apoyan una bipartición, pero también hace crecer enormemente el espacio que debe explorarse.

Los métodos de distancias resumen las columnas del MSA en valores por pares, mientras que los métodos basados en caracteres conservan el patrón de estados de cada columna.

1.8.1 Cuántos árboles pueden proponerse

Para n hojas etiquetadas, el número de topologías bifurcantes no enraizadas es $(2n - 5)!!$, el ^{Referencia} doble factorial de $2n - 5$:

$$N_{\text{no enraizados}}(n) = (2n - 5)!! = (2n - 5)(2n - 7) \cdots 3 \cdot 1. \quad (1.5)$$

Hay 3 topologías bifurcantes no enraizadas para cuatro taxones y 2.027.025 para diez; el crecimiento combinatorio impide una enumeración exhaustiva en conjuntos moderados.

Para cuatro taxones existen exactamente tres biparticiones: $AB|CD$, $AC|BD$ y $AD|BC$. Para diez, el cálculo es $15!! = 15 \cdot 13 \cdot 11 \cdot 9 \cdot 7 \cdot 5 \cdot 3 \cdot 1 = 2.027.025$. Python puede construir el doble factorial con un bucle:

```
def double_factorial(value):
    result = 1
    for factor in range(value, 0, -2):
        result = result * factor
    return result

assert double_factorial(2 * 4 - 5) == 3
assert double_factorial(2 * 10 - 5) == 2_027_025
```

`range(value, 0, -2)` produce una secuencia descendente que resta dos en cada paso y se detiene antes de cero. El guion bajo dentro de `2_027_025` solo mejora la lectura del entero en Python; no altera su valor.

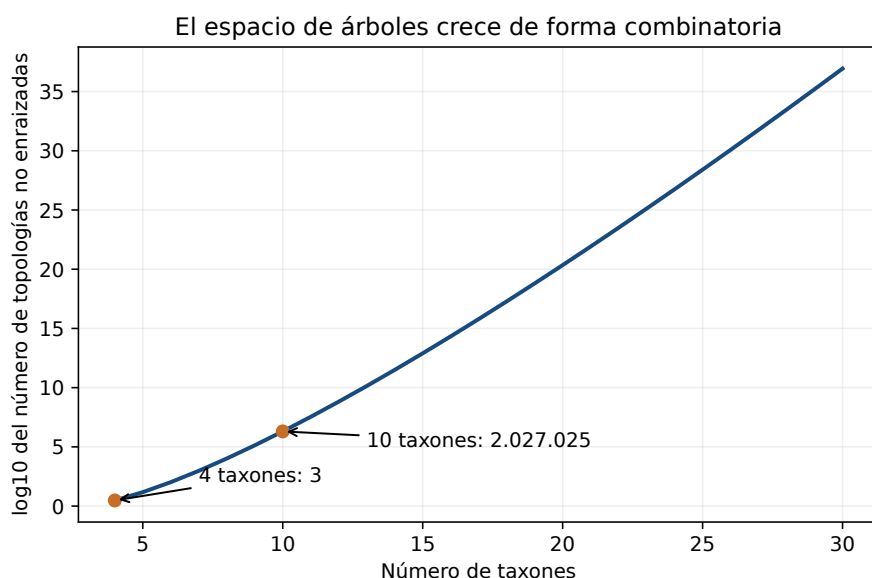


Figure 1.8: Crecimiento del número de topologías bifurcantes no enraizadas. El eje vertical usa logaritmo decimal para que la curva pueda representarse.

Descripción alternativa. El número de topologías no enraizadas crece de 3 con cuatro taxones a 2.027.025 con diez y continúa aumentando de forma combinatoria.

Una búsqueda heurística visita una parte de ese espacio. Suele comenzar en uno o varios árboles iniciales, propone reorganizaciones locales y conserva los cambios que mejoran el criterio. Repetir búsquedas desde puntos diferentes reduce el riesgo de quedar atrapado en un óptimo local, pero no demuestra que se haya encontrado el óptimo global.

Una búsqueda heurística puede terminar en un óptimo local.

Usar varios puntos iniciales amplía la exploración, pero no garantiza haber encontrado el óptimo global.

1.8.2 Parsimonia: contar cambios mínimos

La máxima parsimonia prefiere el árbol que requiere el menor número de cambios para explicar los caracteres. Para una topología fija, el algoritmo de Fitch propaga conjuntos de estados desde las hojas: si los conjuntos hijos se intersectan, conserva la intersección sin sumar cambio; si no se intersectan, conserva la unión y suma uno [9].

En la recurrencia de Fitch, una intersección no vacía de estados hijos se propaga sin sumar cambio; si la intersección es vacía, se propaga la unión y se suma un cambio.

Consideremos cuatro taxones y dos patrones:

Sitio 1: GAAA. Solo A posee G. Es un patrón 3:1, un *singleton*. Requiere un cambio en las tres topologías no enraizadas; no permite escoger entre ellas.

Sitio 2: AAGG. A y B comparten A; C y D comparten G. Requiere un cambio en $AB|CD$ y dos en cada alternativa. Es informativo para parsimonia.

En cuatro taxones, un patrón 3:1 no es informativo para parsimonia porque puntúa igual en las tres topologías; un patrón 2:2 puede favorecer una de ellas.

Podemos ejecutar la lógica sin bibliotecas:

```
def combine(left, right):
    common = left & right
    if common:
        return common, 0
    return left | right, 1

left, score_left = combine({"A"}, {"A"})
right, score_right = combine({"G"}, {"G"})
root, score_root = combine(left, right)
score = score_left + score_right + score_root
assert score == 1
```

El operador `&` calcula intersección de conjuntos y `|` calcula unión. La función devuelve dos valores: el conjunto que sube al nodo y el número de cambios añadido. Para el patrón AAGG en $AB|CD$, ambos pares coinciden internamente y el único cambio se añade al unir $\{A\}$ con $\{G\}$.

En la traza ejecutada del patrón AAGG bajo la agrupación $AB|CD$, ambos pares coinciden internamente y el único cambio añadido se produce al unir $\{A\}$ con $\{G\}$, dando puntuación 1.

Parsimonia es transparente y útil para comprender señal y conflicto. No modela, sin embargo, que varios cambios puedan ocurrir en la misma rama. Bajo ciertas combinaciones de ramas largas separadas y una rama interna corta puede ser estadísticamente inconsistente: al añadir más sitios generados por el mismo proceso sesgado, aumenta el apoyo a una topología incorrecta [4].

Bajo la combinación de dos ramas largas separadas por una rama interna corta estudiada por Felsenstein, máxima parsimonia puede ser estadísticamente inconsistente: al aumentar los sitios generados por ese mismo proceso converge con mayor apoyo hacia una topología incorrecta.

1.8.3 Máxima verosimilitud: probabilidad de los datos bajo una hipótesis

La máxima verosimilitud (ML) pregunta: si fijamos una topología T , sus longitudes de rama y los parámetros θ de un modelo, ¿qué probabilidad tendrían los datos alineados D ? La verosimilitud es una función de la hipótesis con los datos observados fijos:

$$L(T, \theta; D) = P(D \mid T, \theta). \quad (1.6)$$

La máxima verosimilitud compara árboles mediante $P(D \mid T, \theta)$; no calcula directamente la probabilidad de que un árbol sea verdadero.

Si el modelo trata los sitios como independientes condicionados al árbol, la verosimilitud total es el producto de las verosimilitudes de sitio:

$$L(T, \theta; D) = \prod_i P(D_i \mid T, \theta), \quad \ln L = \sum_i \ln P(D_i \mid T, \theta), \quad P(D_i \mid T, \theta) > 0 \text{ para todo } i. \quad (1.7)$$

Bajo independencia condicional de los sitios, la verosimilitud total es el producto de las contribuciones de sitio; si todas son estrictamente positivas, la log-verosimilitud real es la suma de sus logaritmos.

Como el producto de muchas probabilidades muy pequeñas puede desbordar la precisión numérica, se trabaja con logaritmos: el logaritmo de un producto es la suma de los logaritmos. Un árbol con $\ln L = -1200$ tiene mayor verosimilitud que otro con $\ln L = -1250$, porque -1200 es el valor mayor, menos negativo.

Entre dos árboles comparados sobre los mismos datos y criterio, una log-verosimilitud de -1200 es mayor que -1250 y por tanto corresponde a mayor verosimilitud.

Para cada sitio hay estados ancestrales desconocidos. No se elige simplemente el más cómodo: el cálculo suma sobre los posibles estados internos, ponderados por las probabilidades de transición a lo largo de las ramas. El algoritmo de poda de Felsenstein organiza eficientemente esa suma [5, 7]. Aquí basta con entender la estructura; la derivación completa queda en la ruta de referencia del libro.

El algoritmo de poda calcula vectores de verosimilitud condicional desde las hojas hacia la raíz y suma los estados ancestrales ocultos, en vez de enumerar por separado todas sus asignaciones globales.

En un árbol binario de cuatro puntas con patrón (0,0,1,1), probabilidades previas (0,5/0,5), persistencia (0,9) y cambio (0,1), la poda y la enumeración de los tres estados internos producen ambas ($L=0,065700$). Este ejemplo no estima un modelo biológico: es un oráculo pequeño que permite comprobar la recurrencia.

En el ejemplo binario declarado, la poda postorden y la enumeración exhaustiva de estados internos dan la misma verosimilitud, (0,065700).

Tres condicionantes impiden leer «árbol ML» como «historia demostrada»:

1. el modelo puede describir mal composición, tasas o dependencia entre sitios;
2. la búsqueda heurística puede no alcanzar la mejor topología global;
3. diferentes alineamientos o muestras pueden contener señales distintas.

1.8.4 Inferencia bayesiana: incorporar una distribución previa

La inferencia bayesiana trata árboles y parámetros como variables aleatorias. Combina la verosimilitud con una distribución previa y normaliza mediante la probabilidad marginal de los datos:

Estudio

Referencia

Referencia

$$P(T, \theta \mid D) = \frac{P(D \mid T, \theta) P(T, \theta)}{P(D)}.$$

(1.8)

La identidad de Bayes expresa la probabilidad posterior de árbol y parámetros mediante verosimilitud, distribución previa y probabilidad marginal de los datos.

El denominador integra sobre un espacio inmenso y normalmente no puede calcularse por enumeración. Las cadenas de Markov Monte Carlo (MCMC) recorren estados y, después de una fase inicial, aspiran a obtener una muestra de la distribución posterior. Repetir cadenas, inspeccionar trazas, calcular tamaños de muestra efectivos y comprobar convergencia no son adornos: sin ellos, una frecuencia de clado puede reflejar una cadena mal mezclada.

Una estimación posterior mediante MCMC requiere diagnosticar convergencia y suficiencia del muestreo; la identidad de Bayes no garantiza por sí sola que una cadena haya explorado adecuadamente la posterior.

No debe compararse un porcentaje de bootstrap y una probabilidad posterior como si fueran la misma escala. Proceden de procedimientos y preguntas diferentes. Tampoco es correcto describir Bayes como «ML con más precisión»: puede integrar incertidumbre y conocimiento previo, pero depende de priors, modelo y calidad de la exploración [11].

1.8.5 Comparar familias sin convertirlas en una clasificación de calidad

Familia	Entrada	conser- vada	Criterio principal	Límite que debe recordarse
UPGMA	matriz de distan- cias		pareja más cercana y media ponderada	requiere estructura ultra- métrica
NJ	matriz de distan- cias		reducción de longi- tud total	resume columnas y produce árbol no enraizado
Parsimonia	patrones de sitio		mínimo número de cambios	puede sufrir inconsistencia y atracción de ramas largas
ML	patrones de sitio		máxima $P(D \mid T, \theta)$	depende de modelo y búsqueda
Bayes	patrones de sitio		distribución poste- rior	depende además de priors y convergencia MCMC

UPGMA, NJ, parsimonia, ML y Bayes optimizan o resumen objetos distintos; no forman una escala universal en la que el último método sea siempre mejor.

La elección debe empezar por la pregunta y los supuestos. UPGMA es excelente para una traza manual y apropiado para distancias ultramétricas. NJ proporciona un árbol exploratorio rápido sin reloj estricto. Parsimonia muestra qué caracteres apoyan cada alternativa. ML ofrece un marco probabilístico explícito y ampliamente usado. Bayes permite integrar una distribución posterior. En un análisis defendible puede haber varios métodos, pero no se «vota» entre ellos sin estudiar por qué discrepan.

1.9 Modelo, selección y herramienta

1.9.1 Un modelo es una aproximación, no un nombre de menú

JC69 supone simetría extrema. K2P separa transiciones y transversiones. Otros modelos permiten frecuencias desiguales y tasas diferentes; componentes como Γ o categorías de tasa describen heterogeneidad entre sitios. Mayor flexibilidad puede reducir sesgo, pero también exige estimar más parámetros. Elegir el modelo de mayor complejidad disponible no es una regla científica.

Criterios como AIC, AICc o BIC equilibran ajuste y complejidad con penalizaciones distintas. Si ℓ es la log-verosimilitud maximizada, k el número de parámetros y n el número de sitios, se definen aquí como

$$\text{AIC} = -2\ell + 2k, \quad (1.9)$$

$$\text{AICc} = \text{AIC} + \frac{2k(k+1)}{n-k-1}, \quad n > k+1, \quad (1.10)$$

$$\text{BIC} = -2\ell + k \ln n. \quad (1.11)$$

AIC se calcula como $-2\ell + 2k$ con la log-verosimilitud maximizada y el número de parámetros estimados. AICc añade a AIC la corrección $2k(k+1)/(n-k-1)$, que requiere $n > k+1$. BIC se calcula como $-2\ell + k \ln n$.

Por ejemplo, con $\ell = -120$, $k = 5$ y $n = 100$, se obtienen $\text{AIC} = 250,000$, $\text{AICc} = 250,638$ y $\text{BIC} = 263,026$. No son tres estimaciones del mismo parámetro: aplican penalizaciones diferentes y pueden ordenar de modo distinto el mismo conjunto candidato.

ModelFinder evalúa un conjunto de modelos candidatos y permite ordenarlo con AIC, AICc o BIC; está integrado en IQ-TREE [14, 20]. La selección con ModelFinder compara los modelos candidatos bajo el criterio configurado; AIC, AICc y BIC son criterios disponibles y no son sinónimos. Un selector restringido solo puede devolver elementos del conjunto que recibe: si el modelo generador no figura entre los candidatos, no puede seleccionarlo. En el caso mínimo registrado, restringir los candidatos a JC y HKY impide seleccionar el modelo generador GTR+G; al incorporarlo con la menor puntuación, el selector sí devuelve GTR+G. La selección depende del conjunto candidato, el criterio, la versión y las opciones. «El programa eligió GTR+F+G4» es el comienzo de una descripción, no su justificación completa.

Una selección automática de modelo solo es interpretable junto con la versión del software, los modelos candidatos, el criterio y los datos utilizados.

La referencia oficial de órdenes, actualizada el 5 de junio de 2025 y congelada localmente mediante una instantánea normalizada con SHA-256, distingue `-m MF`, que ejecuta la selección extendida sin reconstrucción posterior, de `-m MFP`, que selecciona el modelo y continúa con la reconstrucción del árbol. En la referencia de órdenes de IQ-TREE de 2025, `-m MF` realiza selección extendida de modelo sin reconstrucción posterior del árbol. En esa misma referencia, `-m MFP` realiza selección extendida de modelo y continúa con la reconstrucción del árbol.

La opción `-n 0` omite la búsqueda de árbol posterior a la fase inicial; no solicita por sí misma un árbol NJ o BIONJ. La opción documentada `-t BIONJ` es la que pide usar BIONJ como árbol inicial. En la referencia de órdenes de IQ-TREE de 2025, `-n 0` desactiva la búsqueda posterior del árbol. `-n 0` no es una orden NJ/BIONJ; la referencia documenta separadamente `-t BIONJ` para usar BIONJ como árbol inicial. Las opciones son afirmaciones versionadas: deben comprobarse contra la documentación de la versión ejecutada, no copiarse de una práctica antigua. IQ-TREE 3 amplía el programa con modelos y herramientas filogenómicas, pero ese panorama no cambia el alcance elemental de este capítulo [27, 12].

1.9.2 Una tubería reproducible, no una receta ciega

Una futura práctica puede usar MAFFT 7.526 e IQ-TREE 3, pero Python seguirá siendo el lenguaje principal para validar entradas, registrar versiones, inspeccionar salidas y comparar clados. Bash puede lanzar un ejecutable; no es necesario conocer Bash para comprender la ciencia [19, 12]. La ruta ejecutable de este capítulo usa Python como lenguaje curricular y no presupone Bash para comprender o reproducir sus cálculos. El siguiente bloque es un **fragmento opcional**: construye y muestra una orden, pero no ejecuta IQ-TREE. Para ejecutarla en C5 harán falta IQ-TREE 3 y el archivo versionado `alignment.fasta`; ninguno es prerrequisito de C4.


```

command = [
    "iqtree3", "-s", "alignment.fasta",
    "-m", "MFP", "-B", "1000", "--seed", "20260805"
]
print(command)

```

`command` es una lista: cada elemento corresponde a un argumento, sin necesidad de componer una cadena de shell. El fragmento opcional construye de forma determinista una orden IQ-TREE 3 con entrada, selección de modelo, 1000 réplicas y semilla, pero no afirma haberla ejecutado. Una ejecución posterior tendrá que comprobar el código de retorno y guardar versión, comando, semilla, archivos de entrada y salida.

Una inferencia reproducible conserva identidad y hash de las entradas, versión y parámetros de herramientas, semilla cuando procede, salidas intermedias y controles que relacionan los mismos taxones.

La semilla registra el estado pseudoaleatorio solicitado, pero el protocolo de este proyecto no la acepta como procedencia suficiente. Exige además software, entorno, entradas y parámetros. El capítulo puede impartirse sin conexión mediante instantáneas verificadas; la red se usa para obtener y auditar fuentes, no como requisito durante el aprendizaje.

Como norma de este proyecto, registrar una semilla no satisface por sí solo el contrato de procedencia: también se registran software, entorno, entradas y parámetros.

1.10 Apoyo no significa verdad

1.10.1 Qué remuestrea el bootstrap

El bootstrap filogenético no paramétrico trata las columnas del MSA como la población empírica disponible. Para cada pseudorréplica selecciona columnas al azar con reemplazo hasta recuperar la misma longitud. Algunas aparecen varias veces y otras ninguna. Se reconstruye un árbol por réplica y se cuenta la frecuencia con que aparece cada clado [6].

El bootstrap no paramétrico de sitios remuestrea columnas del alineamiento al azar y con reemplazo hasta igualar la longitud original, reconstruye un árbol por pseudorréplica y cuenta la frecuencia con la que aparece cada clado.

Esta operación trata las columnas como unidades aproximadamente intercambiables bajo el modelo de sitios. Si hay dependencia fuerte entre sitios, bloques ligados o particiones con procesos distintos, la unidad o la estrategia de remuestreo debe respetar esa estructura; barajar columnas como si fueran independientes puede responder a otra pregunta.

El bootstrap estándar de sitios remuestrea columnas como unidades aproximadamente intercambiables; dependencias o particiones fuertes requieren una estrategia de remuestreo coherente con esa estructura.

En el esquema, AB aparece en tres de cuatro árboles, de modo que el apoyo es $3/4 = 0,75 = 75\%$. Cuatro réplicas son insuficientes para un análisis real, pero hacen visible la operación. Si aumentamos el número de réplicas, reducimos el error Monte Carlo del porcentaje; no corregimos automáticamente un MSA sesgado ni un modelo inadecuado.

Aumentar el número de réplicas bootstrap reduce el error Monte Carlo del porcentaje de apoyo, pero no corrige automáticamente un MSA sesgado ni un modelo inadecuado.

Si un clado reaparece con proporción $\hat{p}$ en B pseudorréplicas, una aproximación binomial al error estándar Monte Carlo de esa proporción es

$$SE_{MC}(\hat{p}) = \sqrt{\frac{\hat{p}(1 - \hat{p})}{B}}.$$

MSA original									
	1	2	3	4	5	6	7	8	
R1	1	1	3	4	4	6	8	8	→ árbol → clado AB
R2	2	3	3	5	5	6	7	8	→ árbol → clado AC
R3	1	2	4	4	5	7	7	8	→ árbol → clado AB
R4	1	2	3	4	5	6	7	8	→ árbol → clado AB

Misma longitud; columnas muestreadas con reemplazo

AB aparece en 3 de 4 réplicas: 75 %

Figure 1.9: Ejemplo didáctico de cuatro pseudorréplicas. El 75 % solo describe la frecuencia del clado AB en esta colección mínima.

Descripción alternativa. Cuatro filas muestran índices de columnas entre 1 y 8. Cada fila conserva longitud ocho y algunas repiten índices. Los resultados AB, AC, AB y AB producen un apoyo didáctico de 75 por ciento para AB.

Para $\hat{p} = 0,75$, resulta 0,043301 con $B = 100$ y 0,013693 con $B = 1000$: multiplicar por diez el número de réplicas reduce este error aproximadamente por $\sqrt{10}$, no por diez. Para $\hat{p} = 0,75$, la aproximación binomial produce errores estándar Monte Carlo de 0,043301 con 100 réplicas y 0,013693 con 1000. En la aproximación binomial declarada, aumentar (B) reduce el error Monte Carlo; la fórmula solo contiene (B) y $\hat{p}$, no modifica el MSA ni el modelo que generaron los árboles. La incertidumbre debida al alineamiento y a la adecuación del modelo debe auditarse por separado: aumentar réplicas de bootstrap no sustituye esas comprobaciones.

Esencial

El bootstrap filogenético mide repetibilidad bajo remuestreo de columnas y bajo la tubería elegida; no es la probabilidad de que un clado sea históricamente verdadero.

Un apoyo alto responde aproximadamente: «cuando perturbamos la muestra de columnas de esta manera y repetimos este análisis, el clado reaparece con frecuencia». No responde sin más: «hay un 95 % de probabilidad de que la historia sea cierta». Umbrales como 70 o 95 no son leyes universales; dependen del procedimiento, del contexto y de la decisión que se quiera sostener.

Un apoyo bootstrap alto se interpreta como «al perturbar la muestra de columnas de esta manera y repetir el análisis, el clado reaparece con frecuencia», no como «existe un tanto por ciento de probabilidad de que la historia sea cierta».

Estudio

Los valores 70 y 95 no se aplican como umbrales universales de apoyo: su interpretación exige declarar procedimiento, contexto y decisión.

El bootstrap estándar reconstruye árboles para pseudorréplicas. UFBoot2 emplea una aproximación basada en árboles candidatos para reducir coste y tiene propiedades e interpretación operativa propias [10]. UFBoot2 es un procedimiento aproximado con estrategia de árboles candidatos y no equivale a repetir literalmente una búsqueda estándar completa e independiente para cada pseudorréplica. Informar «bootstrap» sin distinguir el procedimiento deja incompleta la afirmación.

Esencial

1.10.2 Apoyo local y estabilidad global

Un número junto a una rama es local: se refiere al clado definido por esa rama. Un árbol puede contener unos clados muy estables y otros no resueltos. También puede ocurrir que dos análisis mantengan el clado principal pero cambien longitudes o posición del outgroup. La evaluación no debe reducir todo el árbol a un promedio de apoyos.

El apoyo asociado a una rama describe la frecuencia del clado local que esa rama define allí mismo bajo el procedimiento de remuestreo declarado. En el caso ejecutable, dos análisis conservan el clado AB pero difieren tanto en la longitud de sus ramas terminales como en el taxón usado para enraizar. La estabilidad global no se resume mediante el promedio de apoyos de rama: deben informarse por separado los clados comparados, las longitudes y el enraizamiento.

Para comparar dos árboles enraizados pequeños, Python puede representar cada clado como un `frozenset` y calcular intersección y diferencias:

```
clades_raw = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "C"})
}
clades_trimmed = {
    frozenset({"A", "B"}),
    frozenset({"A", "B", "D"})
}

shared = clades_raw & clades_trimmed
only_raw = clades_raw - clades_trimmed
only_trimmed = clades_trimmed - clades_raw

assert shared == {frozenset({"A", "B"})}
```

El ejemplo Python de sensibilidad identifica el clado AB como compartido y conserva por separado los clados exclusivos de los dos análisis.

En el ejemplo registrado, compartir el clado AB no hace idénticos los árboles: uno conserva ABC como clado exclusivo y el otro ABD.

La intersección `&` devuelve clados compartidos; la resta devuelve los exclusivos de cada análisis. Esta comparación no decide cuál es correcto. Localiza qué parte de la conclusión es estable y dónde debe concentrarse la interpretación.

1.11 Sensibilidad: perturbar una decisión con intención

Una prueba de sensibilidad modifica una decisión razonable y observa qué afirmaciones cambian. No consiste en probar opciones hasta obtener el árbol preferido. Antes de ejecutar la perturbación se formula una predicción: «si las columnas ambiguas sostienen el clado X, al retirarlas disminuirá su apoyo»; después se comprueba.

Un análisis de sensibilidad atribuye los cambios comparando tuberías que difieren en una decisión controlada, con una predicción previa y los demás elementos mantenidos constantes.

Perturbaciones útiles:

- comparar dos hipótesis de alineamiento;
- analizar MSA sin recortar y recortado;
- cambiar entre modelos razonables;
- retirar o sustituir el grupo externo;

- excluir temporalmente un taxón de rama muy larga;
- repetir la búsqueda desde semillas o árboles iniciales distintos;
- comparar distancia p y una corrección de modelo dentro de su dominio.

Una matriz de análisis ayuda a no confundir niveles:

Perturbación	Si cambia el resultado	Interpretación prudente
taxones o accesiones	cambia el objeto biológico	revisar ortología, contaminación y cobertura
alineamiento o recorte	cambia la matriz de caracteres	localizar columnas y clados sensibles
modelo	cambia la descripción del proceso	comprobar adecuación y saturación
búsqueda o semilla	cambia la topología óptima hallada	ampliar exploración; no atribuirlo a biología
raíz o outgroup	cambia la polaridad	separar topología no enraizada de historia enraizada

Los errores de datos, alineamiento, modelo, búsqueda y enraizamiento actúan en niveles distintos; una comparación controlada permite localizar, pero no eliminar por sí sola, la incertidumbre.

1.12 Errores y límites que un árbol bonito puede ocultar

1.12.1 Atracción de ramas largas

Esencial

Dos linajes que acumulan muchos cambios pueden compartir estados por convergencia o reversión. Un método que interpreta esos estados como una sola innovación compartida puede agrupar ramas largas que en la historia generadora estaban separadas. Este fenómeno es especialmente conocido para parsimonia en la «zona de Felsenstein», aunque ramas largas también alertan de saturación, modelo insuficiente o muestreo deficiente.

La atracción de ramas largas puede agrupar linajes distantes cuando cambios múltiples y homoplasias producen semejanzas que el criterio interpreta como historia compartida.

En F09, las ramas terminales de A y D miden tres unidades del esquema y las de B y C una unidad; la razón visual larga:corta es 3:1.

La respuesta no es borrar siempre la secuencia incómoda. Primero se comprueba su identidad, calidad y alineamiento; se amplía el muestreo si es posible; se examina saturación y composición; se usan modelos más apropiados; y se compara el resultado con y sin el taxón como diagnóstico. Eliminarlo definitivamente requiere una razón biológica o técnica documentada.

Estudio

Una rama larga no justifica por sí sola borrar un taxón: el protocolo exige revisar identidad, calidad, alineamiento, saturación y modelo, y usar la comparación con/sin taxón como diagnóstico documentado.

Esencial

1.12.2 Poca señal y conflicto no son lo mismo

Un MSA puede tener pocas columnas variables: casi todos los árboles reciben información insuficiente. También puede contener muchas columnas variables que apoyan historias incompatibles. En el primer caso hay ausencia de señal; en el segundo, conflicto. Un consenso poco resuelto puede ser una representación honesta de ambos, pero las acciones posteriores difieren: añadir caracteres homólogos puede ayudar ante poca señal; ante conflicto real quizá haga falta un modelo de procesos múltiples.

Atracción de ramas largas: la longitud es una magnitud, no una etiqueta

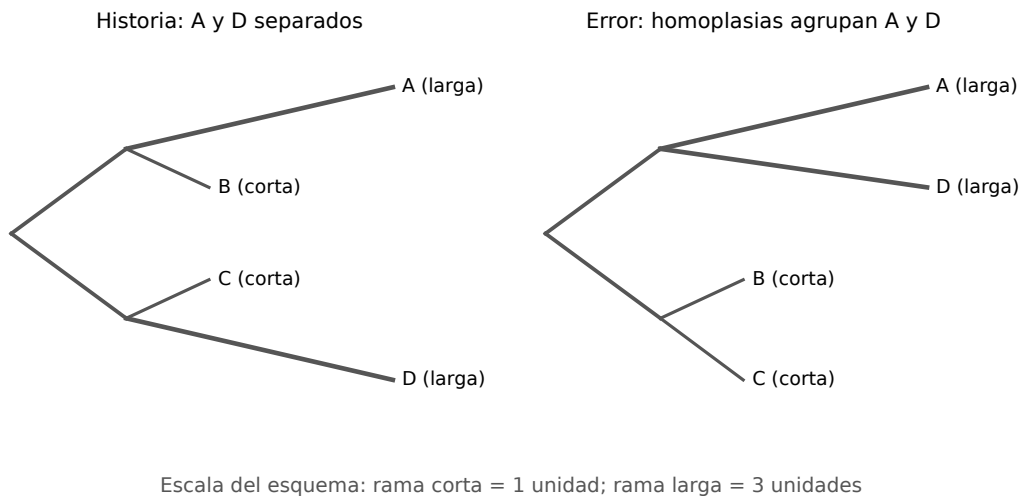


Figure 1.10: Esquema conceptual de atracción de ramas largas. No representa una simulación empírica ni afirma que toda rama larga sea artefactual.

Descripción alternativa. El panel izquierdo separa A y D y sus ramas terminales son exactamente tres veces más largas que las de B y C en la escala del esquema. El panel derecho muestra el error conceptual: A y D quedan agrupados por estados coincidentes, no porque toda rama larga deba agruparse.

Poca señal y conflicto no son equivalentes: la primera aporta información insuficiente, mientras el segundo contiene caracteres que apoyan historias incompatibles y puede requerir modelos de procesos múltiples.

1.12.3 Composición, tasas y dependencia entre sitios

Muchos modelos básicos suponen un proceso homogéneo a lo largo de ramas, estacionario en composición y reversible. Si dos linajes adquieren composiciones parecidas por sesgos independientes, pueden agruparse artificialmente. Si los sitios interactúan por estructura, la independencia es una aproximación. Si las tasas cambian entre ramas o a través del tiempo, un modelo uniforme puede fallar. Los supuestos explícitos son valiosos porque pueden auditarse; no porque se vuelvan ciertos al escribirlos.

Composición heterogénea, tasas variables entre ramas o dependencia entre sitios pueden violar supuestos de modelos homogéneos, estacionarios o independientes y sesgar la inferencia.

1.12.4 Recombinación y mosaicos de historia

Un solo árbol supone que los caracteres analizados comparten una historia arbórea suficientemente común. La recombinación puede hacer que regiones distintas tengan genealogías diferentes. Concatenarlas produce una media cuya interpretación depende del objetivo y del modelo. En este capítulo basta con reconocer la señal de alarma: cambios bruscos de apoyo entre regiones o discordancia persistente no deben ocultarse con un único árbol.

La recombinación puede producir genealogías diferentes entre regiones, de modo que un único árbol para una secuencia mosaico puede resumir historias incompatibles.

Concatenar regiones con genealogías distintas produce una señal promediada cuya interpretación depende del objetivo del análisis y del modelo empleado.

Un cambio brusco de apoyo entre regiones o una discordancia persistente al concatenar son señales de alarma que no deben ocultarse detrás de un único árbol resumen.

1.12.5 Etiquetas y procedencia

Una etiqueta incorrecta puede ser más dañina que un parámetro subóptimo. Cada secuencia debe relacionarse con organismo, gen o proteína, accesión y versión. No se reutiliza una misma accesión para organismos distintos salvo que la fuente lo explique. Antes de inferir:

1. comprobar número de registros y etiquetas únicas;
2. verificar que todas las secuencias pertenecen al tipo esperado;
3. revisar longitudes y caracteres inválidos;
4. confirmar que MSA, matriz y árbol contienen los mismos taxones;
5. calcular hashes de los archivos distribuidos.

La procedencia no es burocracia posterior. Define qué entidades biológicas representan las hojas.

El protocolo de entrada relaciona cada secuencia con una etiqueta, accesión y versión, tipo molecular y procedencia antes de inferir. Los controles ejecutables rechazan etiquetas duplicadas, tipo molecular inesperado y caracteres inválidos para el alfabeto declarado. MSA, matriz y árbol deben conservar exactamente el mismo conjunto de etiquetas taxonómicas; el fixture registrado comprueba esa igualdad. El control ejecutable rechaza una accesión asignada a dos organismos distintos cuando el registro no aporta una justificación explícita de esa reutilización. El control ejecutable resume la longitud mínima y máxima de las secuencias aceptadas y rechaza una secuencia vacía antes de la inferencia.

Estudio

1.13 Python como cuaderno verificable

Python no reemplaza el razonamiento filogenético; lo hace inspeccionable. En este capítulo se han utilizado pocas construcciones, reintroducidas donde aparecen [23]:

Estudio

Variables. Nombres que apuntan a valores, como `distance_ab_d = 32.5`.

Cadenas. Texto entre comillas, usado para secuencias y taxones.

Listas. Colecciones ordenadas entre corchetes, adecuadas para argumentos o columnas.

Diccionarios. Asociaciones clave-valor entre llaves, adecuadas para matrices etiquetadas.

Conjuntos. Colecciones sin orden ni duplicados, adecuadas para clados.

Funciones. Operaciones reutilizables definidas con `def`.

Condiciones. Decisiones expresadas con `if`.

Bucles. Repeticiones expresadas con `for`.

Excepciones. Errores explícitos creados con `raise`.

Aserciones. Propiedades exigidas con `assert`.

En Python, una lista conserva orden y duplicados, un diccionario recupera valores por clave y un conjunto conserva elementos únicos sin imponer un orden curricular. Una función definida con `def` puede combinar condiciones y bucles; `raise` comunica una entrada inválida y `assert` detiene la ejecución si una propiedad exigida es falsa.

Una buena comprobación separa tres capas:

```
import math

def jc69(p):
    if p < 0 or p >= 0.75:
        raise ValueError("JC69 exige 0 <= p < 0.75")
    return -0.75 * math.log(1 - 4 * p / 3)

# 1. Entrada reconstruida y explícita
p = 0.10

# 2. Transformación
d = jc69(p)

# 3. Propiedad esperada
assert d > p
assert abs(d - 0.107326) < 1e-6
```

El ejemplo de cuaderno verificable es autocontenido: define JC69, fija la entrada y comprueba una propiedad cualitativa y el valor numérico.

El primer `assert` comprueba una propiedad cualitativa del ejemplo: la corrección supera la distancia observada. El segundo comprueba el valor publicado con tolerancia. Si se cambia la entrada, debe revisarse también la expectativa; copiar una salida antigua sería perpetuar un resultado sin procedencia.

1.13.1 Bibliotecas: conveniencia sin ocultar el objeto

Para árboles mayores, `Bio.Phylo.read` puede leer un único árbol Newick y devolver un objeto recorrible [3, 25, 2]. El caso ejecutable del capítulo consulta la versión instalada en vez de presuponerla, lee una entrada mínima y recupera las hojas A, B y C. Antes de usar una biblioteca conviene saber qué objeto esperamos: hojas, clados, raíz y longitudes. La biblioteca facilita el recorrido, pero no decide si una secuencia es ortóloga ni si una raíz está justificada.

La ejecución controlada consulta la versión instalada de Biopython, llama realmente a `Bio.Phylo.read` sobre un Newick mínimo y recupera exactamente las hojas A, B y C; además rechaza una entrada vacía y una expectativa de hojas incorrecta.

La extensión opcional de C5 fijará Biopython 1.87 y proporcionará un Newick mínimo en memoria, sin descarga ni archivo oculto. Esa versión futura no se atribuye a la ejecución C4: el resultado presente publica la versión realmente observada. El comprobador central `verification/chapter_checks.py` declara Python 3.9 o posterior y sus importaciones se limitan a la biblioteca estándar y al fixture local `fixtures.py`.

Como contrato de la futura extensión C5, Biopython se fijará en la versión 1.87 y el Newick será autocontenido en memoria; la extensión no se presenta como ejecutada en C4. El archivo `verification/chapter_checks.py` se analiza sintácticamente en ejecución, exige Python 3.9 como mínimo e importa solo módulos de la biblioteca estándar y el fixture local `fixtures.py`.

1.14 Un caso integrado: de la pregunta a una afirmación limitada

Supongamos que queremos estudiar una familia de globinas. El objetivo no es «hacer un árbol de mamíferos», sino distinguir copias alfa y beta y comprobar si las copias de diferentes especies

se agrupan según la duplicación ancestral. La cadena defendible sería:

1. **Pregunta.** ¿La separación alfa-beta antecede a las especiaciones incluidas?
2. **Selección.** Reunir accesiones verificadas de alfa y beta en varias especies, con una copia externa justificada.
3. **Predicción.** Si la duplicación es anterior, esperamos dos clados principales de copias, no un clado exclusivo por especie.
4. **MSA.** Alinear proteínas, inspeccionar regiones ambiguas y conservar una alternativa razonable.
5. **Modelo y búsqueda.** Inferir un árbol ML bajo selección de modelo registrada y búsquedas reproducibles.
6. **Apoyo.** Estimar apoyo de las biparticiones y comprobar sensibilidad al tratamiento del MSA.
7. **Afirmación.** «En este muestreo y bajo estas decisiones, la separación de copias alfa y beta es estable»; no «este es el árbol definitivo de los mamíferos».

Un segundo conjunto, compuesto por ortólogos verificados de TP53 en vertebrados, respondería a una pregunta distinta. Mantener separados ambos casos evita usar una mezcla de parálogos para inferir directamente relaciones de especies.

1.15 Síntesis: qué puede defenderse y qué queda abierto

Una inferencia filogenética elemental es defendible cuando podemos recorrerla en ambas direcciones. Hacia delante: pregunta, datos, MSA, modelo, método, apoyo y conclusión. Hacia atrás: cada conclusión remite a un clado; cada clado a árboles y pseudorréplicas; cada árbol a un MSA y un modelo; cada MSA a secuencias identificadas.

Esencial

Las ideas nucleares son:

- leer conexiones, no posiciones de la página;
- separar topología, raíz y longitud de rama;
- distinguir similitud de homología y ortología de paralogía;
- tratar el MSA como hipótesis de homología posicional;
- interpretar una corrección de distancia dentro de su dominio;
- ponderar UPGMA por tamaños de clúster y reconocer su reloj;
- comparar métodos por objeto, criterio y supuesto;
- interpretar apoyo como estabilidad condicionada;
- perturbar una decisión para localizar sensibilidad;
- no identificar un árbol génico con un árbol de especies.

La pregunta de apertura puede responderse ahora: pasamos de secuencias comparables a una hipótesis evolutiva defendible solo si conservamos la cadena de decisiones y sus límites. El árbol no cierra la investigación. Organiza qué relaciones merecen una prueba posterior, qué datos deben ampliarse y qué conclusiones todavía no pueden sostenerse.

Ampliación y fuentes de consulta

La síntesis local de Abascal, Irisarri y Zardoya ofrece una introducción en castellano a evolución molecular, lectura de árboles y familias de métodos [1]. Para lectura crítica de árboles son especialmente útiles Omland y colaboradores [22]. Los fundamentos clásicos de distancias, NJ, parsimonia, ML y bootstrap se encuentran en las fuentes originales citadas en sus secciones [13, 16, 24, 9, 5, 6]. La terminología original de ortología y paralogía puede consultarse en Fitch [8].

La bibliografía reciente no se usa para inflar el capítulo, sino para cerrar lagunas concretas: riesgos del filtrado de MSA [26], discordancia gen-especie [17], selección de modelos [14] y procedimientos actuales de inferencia y apoyo [27, 10]. Las instrucciones ejecutables deben contrastarse además con la documentación oficial de las versiones instaladas.

Puente al anexo práctico. El anexo recuperará las secciones 1.2, 1.4, 1.5, 1.6 y 1.10. Allí se reproducirán primero los cálculos ancla sin IA y después con Python; solo entonces se introducirá una perturbación controlada. La práctica no volverá a enseñar estas definiciones.

Anexo práctico · De secuencias a una hipótesis filogenética

Pregunta, producto y separación de materiales

Dependencias estables: BC09-SEC-TREES, BC09-SEC-MSA, EQ-BC09-JC69, BC09-SEC-UPGMA y BC09-SEC-BOOTSTRAP.

Cuatro secuencias sintéticas, ya alineadas, plantean la pregunta que organiza este anexo: *¿qué taxón se declara más próximo, con qué distancia, y qué parte de esa decisión procede de los datos y cuál del modelo de corrección?*

El anexo usa un conjunto propio de cuatro secuencias sintéticas ajeno al ejemplo A-B-C-D de la teoría, para que el cálculo ancla no pueda copiarse del cuerpo teórico.

El producto individual contiene: la respuesta manual del cálculo ancla; la salida de `verification/practice_checks.py` para las cuatro variantes (árbol base, perturbación de modelo, perturbación de datos, control de dominio); el resultado del comprobador público sobre los valores propios; y una defensa escrita de 150–250 palabras. El PDF del estudiante no contiene la clave docente ni los valores numéricos resueltos de la corrección JC69; la clave permanece en `instructor_key/` y no se enlaza desde este anexo.

Teoría que se recupera. Use solo cuatro referencias: BC09-SEC-TREES para leer el árbol resultante sin añadir historia que no contiene; EQ-BC09-JC69 para la fórmula y su dominio; BC09-SEC-UPGMA para la regla de actualización ponderada; y BC09-SEC-BOOTSTRAP solo como referencia de que un único árbol no certifica verdad histórica, sin ejecutar remuestreo en este anexo. BC09-SEC-MSA se recupera únicamente para recordar por qué las cuatro secuencias ya vienen alineadas y sin huecos: aísla el problema de distancias del problema de alineamiento, que no es objeto de este anexo.

0 · Entorno y diagnóstico

Python 3.9 o superior, sin paquetes externos. Verifique antes de empezar:

```
python3 --version
sha256sum --check --strict data/manifest.sha256
python3 verification/practice_checks.py domain_guard
```

El tercer comando debe imprimir `domain_guard=REJECTED p=0.8 out_of_domain=True`. Si falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto; no continúe hasta resolverlo.

Si el diagnóstico inicial de entorno falla, el entorno no tiene el intérprete esperado o los archivos no coinciden con el manifiesto de hashes; el anexo exige resolverlo antes de continuar con el cálculo ancla.

La evidencia mínima entregable es el cálculo ancla a mano, el script Python ejecutado con sus cuatro salidas y una defensa escrita de 150 a 250 palabras; el ejercicio funciona sin red y sin más dependencia que Python 3.9 estándar.

1 · Cálculo ancla, sin IA

Abra `data/practice_sequences.fasta`. Contiene S1–S4, 16 nt cada una, ya alineadas. Trabaje solo con calculadora o lápiz y papel.

Tarea. Antes de usar IA, calcula a mano una distancia corregida y la primera actualización ponderada de UPGMA. Reprodúcelas después con un programa Python completo y legible. Perturba una secuencia o una decisión de alineamiento, vuelve a ejecutar el análisis y separa qué cambio procede de los datos, del modelo y del algoritmo.

Concretamente, entregue:

1. el número de posiciones en que difieren S1 y S2, y la p-distancia correspondiente;
2. la distancia JC69 corregida de ese par, aplicando la fórmula de EQ-BC09-JC69 y comprobando que la p-distancia está en su dominio;
3. la altura de la primera fusión UPGMA, que para dos hojas es la distancia corregida dividida entre dos;
4. la primera actualización ponderada: dadas $d(S1, S3) = 0,404247$ y $d(S2, S3) = 0,656602$ (calculadas de la misma manera que el punto 2, para no repetir dos veces el mismo cálculo a mano), la distancia del clúster $\{S1, S2\}$ a S3 pondera por tamaño de clúster: con dos hojas de partida, ambos pesos valen 1, pero la fórmula es la misma que se usará después con pesos distintos.

En el conjunto propio del anexo, S1 y S2 difieren en 2 de 16 posiciones ($p=0,125$) y su distancia JC69 corregida es 0,136741.

Compruebe su resultado sin revelar la solución completa:

```
python3 verification/practice_public_checker.py \
  --diffs <su_conteo> --p <su_p> --jc69 <su_distancia> \
  --d13 0.404247 --d23 0.656602 --updated <su_actualizacion>
```

Un FAIL señala qué relación algebraica no se cumple; no imprime el valor correcto. Si falla `formula_jc69`, revise la fórmula, no vuelva a adivinar un número.

2 · Reproducción en Python

Con el cálculo ancla ya resuelto a mano, ejecute la reproducción completa:

```
python3 verification/practice_checks.py anchor
python3 verification/practice_checks.py full_tree
```

El primer comando debe coincidir con sus cuatro valores manuales dentro de 10^{-4} . El segundo construye el árbol UPGMA completo sobre las cuatro secuencias con distancias JC69, fusión a fusión, incluidas las fusiones que usted no calculó a mano.

El árbol UPGMA ponderado completo sobre S1-S4 con distancias JC69 funde primero S1 con S2, después incorpora S3 y finalmente S4 en la raíz, a una altura de 0,534583.

El andamiaje se retira aquí: el paso 1 le dio los tres primeros números; el paso 2 no repite la fórmula en el enunciado, solo pide leer la salida y contrastarla contra lo que usted ya obtuvo a mano.

3 · Perturbación: modelo frente a datos

Ejecute las dos perturbaciones y compare ambas salidas con la del paso 2:

```
python3 verification/practice_checks.py perturb_model
python3 verification/practice_checks.py perturb_data
python3 verification/practice_checks.py invariant_first_merge
```

`perturb_model` repite el mismo árbol usando p-distancia sin corregir en vez de JC69: cambia el modelo, no los datos. `perturb_data` aplica dos sustituciones declaradas en S3 (posiciones 2 y 9, ambas $G \rightarrow A$) y vuelve a calcular con JC69: cambia los datos, no el modelo. `invariant_first_merge` comprueba automáticamente si S1+S2 sigue siendo la primera fusión en las tres variantes.

Sustituir las distancias JC69 por p-distancias sin corregir conserva el orden de fusión de este conjunto, pero reduce todas las alturas.

Introducir dos sustituciones adicionales en S3 cambia el orden de fusión: S4 pasa a incorporarse antes que S3, que se convierte en el taxón más divergente.

La pareja S1-S2 se funde primero en las cuatro variantes calculadas (base, corrección JC69, perturbación de modelo y perturbación de datos), mientras que el resto de la topología responde de forma distinta a cada tipo de perturbación.

No dé por sentado que ninguna perturbación cambiaría la topología: compruebe las dos por separado. Que una no la cambie en este conjunto no demuestra que el modelo nunca importa; demuestra solo que, para estos cuatro taxones y esta sustitución de dos bases, la corrección afectó la escala y no el orden.

4 · Controles

- **Taxones coincidentes.** Las cuatro variantes deben devolver exactamente los taxones S1, S2, S3, S4; ninguno se pierde ni se duplica.
- **Matriz simétrica y diagonal nula.** Para cualquier par, la distancia calculada en un sentido debe coincidir con la del sentido contrario, y la distancia de una secuencia consigo misma es cero.
- **Dominio de la corrección.** `python3 verification/practice_checks.py domain_guard` confirma que una $p \geq 0,75$ se rechaza en vez de devolver un número.
- **Cordura del algoritmo.** El primer par fundido siempre debe ser el de menor distancia entre los disponibles en ese paso; verifique esta propiedad en al menos una fusión que no sea la primera.

La implementación de JC69 usada en el anexo rechaza explícitamente una p-distancia fuera de dominio ($p \geq 0,75$) en vez de devolver un valor numérico engañoso.

5 · Interpretación, límites y defensa

Escriba una defensa de 150–250 palabras que separe explícitamente:

1. qué procede de los **datos** (la perturbación de S3);
2. qué procede del **modelo** (JC69 frente a p-distancia);

3. qué procede del **algoritmo** (la regla de actualización ponderada, que no cambia entre variantes);
4. qué permanece invariante en las cuatro variantes y por qué eso no prueba una ley general sobre este método con cualquier conjunto de datos.

Límite explícito: con cuatro taxones y una sola perturbación por tipo, este anexo no permite concluir cuál de los dos, modelo o datos, es «más importante» en general; solo permite documentar, para este conjunto, cuál de los dos cambió la topología y cuál no.

6 · Modo de IA y criterios de evidencia

El anexo permite usar IA solo para depurar sintaxis Python ya escrita por el estudiante, nunca para calcular la distancia, decidir la topología o redactar la defensa; existe una vía completa sin IA basada en calculadora y el comprobador público.

Si no dispone de acceso a un asistente, la vía completa sin IA es: calculadora para el paso 1, la reproducción Python del paso 2 ejecutada localmente, y el comprobador público del paso 1 para autoevaluarse antes de entregar. Ninguna parte de este anexo depende de un servicio de red.

Puente desde la teoría. El anexo recupera las secciones declaradas al inicio. No vuelve a enseñar la fórmula de JC69 ni la regla de actualización de UPGMA; las aplica sobre datos nuevos y pide separar qué parte del resultado depende de cada decisión.

Bibliography

- [1] Federico Abascal, Iker Irisarri, and Rafael Zardoya. Filogenia y evolución molecular. In Álvaro Sebastián and Alberto Pascual-García, editors, *Bioinformática con Ñ. Volumen I: Principios de Bioinformática*, pages 231–257. CreateSpace, 2014. Capítulo 9; copia local de consulta.
- [2] Biopython Project. Bio.phylo api documentation. Biopython 1.87 official documentation, 2026. Consulted 2026-08-05; optional C5 extension.
- [3] Peter J. A. Cock, Tiago Antao, Jeffrey T. Chang, et al. Biopython: freely available python tools for computational molecular biology and bioinformatics. *Bioinformatics*, 25(11):1422–1423, 2009.
- [4] Joseph Felsenstein. Cases in which parsimony or compatibility methods will be positively misleading. *Systematic Zoology*, 27(4):401–410, 1978.
- [5] Joseph Felsenstein. Evolutionary trees from dna sequences: a maximum likelihood approach. *Journal of Molecular Evolution*, 17:368–376, 1981.
- [6] Joseph Felsenstein. Confidence limits on phylogenies: an approach using the bootstrap. *Evolution*, 39(4):783–791, 1985.
- [7] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, 2004.
- [8] Walter M. Fitch. Distinguishing homologous from analogous proteins. *Systematic Zoology*, 19(2):99–113, 1970.
- [9] Walter M. Fitch. Toward defining the course of evolution: minimum change for a specific tree topology. *Systematic Zoology*, 20(4):406–416, 1971.
- [10] Diep Thi Hoang, Olga Chernomor, Arndt von Haeseler, Bui Quang Minh, and Le Sy Vinh. Ufboot2: improving the ultrafast bootstrap approximation. *Molecular Biology and Evolution*, 35(2):518–522, 2018.
- [11] Mark Holder and Paul O. Lewis. Phylogeny estimation: traditional and bayesian approaches. *Nature Reviews Genetics*, 4:275–284, 2003.
- [12] IQ-TREE Project. Iq-tree command reference. Official documentation; updated 2025-06-05, 2025. Consulted 2026-08-05.
- [13] Thomas H. Jukes and Charles R. Cantor. Evolution of protein molecules. In *Mammalian Protein Metabolism*, pages 21–132. Academic Press, 1969.
- [14] Subha Kalyaanamoorthy, Bui Quang Minh, Thomas K. F. Wong, Arndt von Haeseler, and Lars S. Jermini. Modelfinder: fast model selection for accurate phylogenetic estimates. *Nature Methods*, 14:587–589, 2017.

- [15] Kazutaka Katoh and Daron M. Standley. Mafft multiple sequence alignment software version 7: improvements in performance and usability. *Molecular Biology and Evolution*, 30(4):772–780, 2013.
- [16] Motoo Kimura. A simple method for estimating evolutionary rates of base substitutions through comparative studies of nucleotide sequences. *Journal of Molecular Evolution*, 16:111–120, 1980.
- [17] Laura Kubatko. An evolving view of species tree inference. *Systematic Biology*, 75(2):195–207, 2026.
- [18] Wayne P. Maddison. Gene trees in species trees. *Systematic Biology*, 46(3):523–536, 1997.
- [19] MAFFT Project. Mafft version 7: official software documentation. Version 7.526; official project site; released April 2024, 2024. Consulted 2026-08-05.
- [20] Lam-Tung Nguyen, Heiko A. Schmidt, Arndt von Haeseler, and Bui Quang Minh. Iq-tree: a fast and effective stochastic algorithm for estimating maximum-likelihood phylogenies. *Molecular Biology and Evolution*, 32(1):268–274, 2015.
- [21] Laura R. Novick and Kefyn M. Catley. When relationships depicted diagrammatically conflict with prior knowledge: an investigation of students’ interpretations of evolutionary trees. *Science Education*, 98(2):269–304, 2014.
- [22] Kevin E. Omland, Lyn G. Cook, and Michael D. Crisp. Tree thinking for all biology: the problem with reading phylogenies as ladders of progress. *BioEssays*, 30(9):854–867, 2008.
- [23] Python Software Foundation. The python tutorial. Python 3.9 official documentation, 2020. Versioned reference for data structures, control flow and exceptions; consulted 2026-08-06.
- [24] Naruya Saitou and Masatoshi Nei. The neighbor-joining method: a new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.
- [25] Eric Talevich, Brandon M. Invergo, Peter J. A. Cock, and Brad A. Chapman. Bio.phylo: a unified toolkit for processing, analyzing and visualizing phylogenetic trees in biopython. *BMC Bioinformatics*, 13:209, 2012.
- [26] Ge Tan, Matthieu Muffato, Christian Ledergerber, Javier Herrero, Nick Goldman, Marcel Gil, and Christophe Dessimoz. Current methods for automated filtering of multiple sequence alignments frequently worsen single-gene phylogenetic inference. *Systematic Biology*, 64(5):778–791, 2015.
- [27] Thomas K. F. Wong, Nhan Ly-Trong, Huaiyan Ren, Piyumal Demotte, Hector Baños, et al. Iq-tree 3: phylogenomic inference software using complex evolutionary models. *Molecular Biology and Evolution*, 43(5):msag117, 2026.